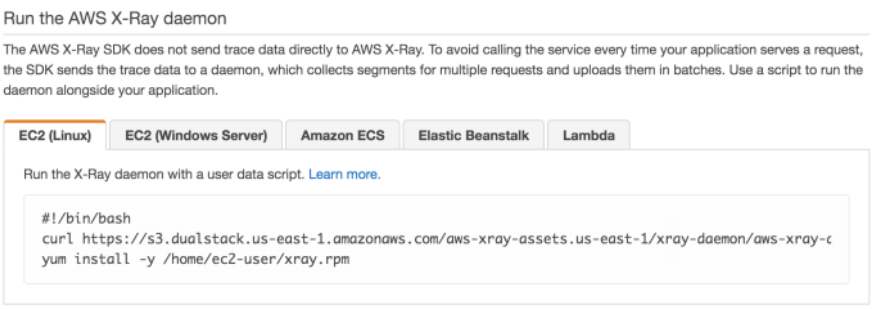
Section-Based – Troubleshooting and Optimization (CDA)

1. **1. Question**
Category: CDA – Troubleshooting and Optimization
In the next financial year, a company has decided to develop a completely new version of its legacy application that will uti lize Node.js and GraphQL. The new architecture aims to offer an end-to-end view of requests as they traverse the application and display a map of the underlying components.
To achieve this, the application will be hosted in an Auto Scaling group (ASG) of Linux EC2 instances behind an Application L oad Balancer (ALB) and must be instrumented to send trace data to the AWS X-Ray.
Which of the following options is the MOST suitable way to satisfy this requirement?

- Refactor your application to send segment documents directly to X-Ray by using the `PutTraceSegments` API.
- Enable AWS X-Ray tracing on the ASG's launch template.
- Enable AWS Web Application Firewall (WAF) on the ALB to monitor web requests.
- Use a user data script to install the X-Ray daemon.

Correct
The AWS X-Ray SDK does not send trace data directly to AWS X-Ray. To avoid calling the service every time your application serves a request, the SDK sends the trace data to a daemon, which collects segments for multiple requests and uploads them in batches. Use a script to run the daemon alongside your application.
To properly instrument your application hosted in an EC2 instance, you have to install the X-Ray daemon by using a user data script. This will install and run the daemon automatically when you launch the instance. To use the daemon on Amazon EC2, create a new instance profile role or add the managed policy to an existing one. This will grant the daemon permission to upload trace data to X-Ray.



The AWS X-Ray daemon is a software application that listens for traffic on UDP port 2000, gathers raw segment data, and relays it to the AWS X-Ray API. The daemon works in conjunction with the AWS X-Ray SDKs and must be running so that data sent by the SDKs can reach the X-Ray service.
Hence, the correct answer is: **Use a user data script to install the X-Ray daemon.**
The option that says: **Enable AWS X-Ray tracing on the ASG's launch template** is incorrect. There's no option to enable X-Ray tracing in a launch template of an ASG.
The option that says: **Enable AWS Web Application Firewall (WAF) on the ALB to monitor web requests** is incorrect. Although it can help monitor and protect the application from common web exploits, it's not capable of instrumenting the application.
The option that says: **Refactor your application to send segment documents directly to X-Ray by using the `PutTraceSegments` API** is incorrect. Although this solution will work, it entails a lot of manual effort to perform. You don't need to do this because you can just install the X-Ray daemon on the instance to automate this process.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-ec2.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon.html#xray-daemon-permissions

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

2. **2. Question**
Category: CDA – Troubleshooting and Optimization
An API gateway with a Lambda proxy integration takes a long time to complete its processing. There were also occurrences wher e some requests timed out. You want to monitor the responsiveness of your API calls as well as the underlying Lambda function.
Which of the following CloudWatch metrics should you use to troubleshoot this issue? (Select TWO.)

- CacheHitCount
- CacheMissCount
- Count
- Latency
- IntegrationLatency

Correct
You can monitor API execution using CloudWatch, which collects and processes raw data from API Gateway into readable, near-real-time metrics. These statistics are recorded for a period of two weeks so that you can access historical information and gain a better perspective on how your web application or service is performing. By default, API Gateway metric data is automatically sent to CloudWatch in one-minute periods.
The metrics reported by API Gateway provide information that you can analyze in different ways. The list below shows some common uses for the metrics. These are suggestions to get you started, not a comprehensive list.
– Monitor the **IntegrationLatency** metrics to measure the responsiveness of the backend.
– Monitor the **Latency** metrics to measure the overall responsiveness of your API calls.
– Monitor the **CacheHitCount** and **CacheMissCount** metrics to optimize cache capacities to achieve a desired performance.
Hence, the correct metrics that you have to use in this scenario are **Latency** and **IntegrationLatency.**
Count is incorrect because this metric simply gets the total number of API requests in a given period.
CacheMissCount is incorrect because this metric just gets the number of requests served from the backend in a given period when API caching is enabled. The `Sum` statistic represents this metric, namely, the total count of the cache misses in the given period.
CacheHitCount is incorrect because this fetches the number of requests served from the API cache in a given period. The `Sum` statistic represents this metric, namely, the total count of the cache hits in the given period.

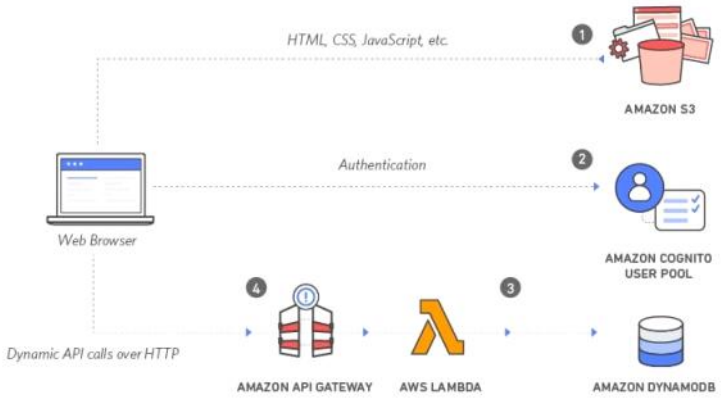
References:
https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-metrics-and-dimensions.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/monitoring-cloudwatch.html

Check out this Amazon API Gateway Cheat Sheet:
https://tutorialsdojo.com/amazon-api-gateway

3. **3. Question**
Category: CDA – Troubleshooting and Optimization
A serverless application, which uses a Lambda function integrated with API Gateway, provides data to a front-end application written in ReactJS. The users are complaining that they are getting HTTP 504 errors intermittently when they are using the application in peak times. The developer found no errors in the CloudWatch logs of the Lambda function.
Which of the following is the MOST likely cause of this issue?

- The memory allocated for the Lambda function is insufficient
- The underlying Lambda function has been running for more than 29 seconds causing the API Gateway request to time out.
- There is an authorization failure occurring between API Gateway and the Lambda function.
- The API Gateway automatically enabled throttling in peak times which caused the HTTP 504 errors.

Correct
A gateway response is identified by a response type defined by API Gateway. The response consists of an HTTP status code, a set of additional headers that are specified by parameter mappings, and a payload that is generated by a non-VTL (Apache Velocity Template Language) mapping template.



You can set up a gateway response for a supported response type at the API level. Whenever API Gateway returns a response of the type, the header mappings and payload mapping templates defined in the gateway response are applied to return the mapped results to the API caller.
The following are the Gateway response types which are associated with the HTTP 504 error in API Gateway:
INTEGRATION_FAILURE – The gateway response for an integration failed error. If the response type is unspecified, this response defaults to the DEFAULT_5XX type.
INTEGRATION_TIMEOUT – The gateway response for an integration timed-out error. If the response type is unspecified, this response defaults to the DEFAULT_5XX type.
For the integration timeout, the range is from 50 milliseconds to 29 seconds for all integration types, including Lambda, Lambda proxy, HTTP, HTTP proxy, and AWS integrations.
In this scenario, there is an issue where the users are getting HTTP 504 errors in the serverless application. This means the Lambda function is working fine at times, but there are instances when it throws an error. Based on this analysis, the most likely cause of the issue is the **INTEGRATION_TIMEOUT** error since you will only get an **INTEGRATION_FAILURE** error if your AWS Lambda integration does not work at all in the first place.
Hence, the correct answer is: **The underlying Lambda function has been running for more than 29 seconds causing the API Gateway request to time out.**
The option that says: **The memory allocated for the Lambda function is insufficient** is incorrect. The fact that no errors were found in the CloudWatch Logs suggests that the function is not the bottleneck.
The option that says: **The API Gateway automatically enabled throttling in peak times which caused the HTTP 504 errors** is incorrect because a large number of incoming requests will most likely produce an HTTP 502 or 429 error but not a 504 error. If executing the function would cause you to exceed a concurrency limit at either the account level (*ConcurrentInvocationLimitExceeded*) or function level (*ReservedFunctionConcurrentInvocationLimitExceeded*), Lambda may return a *TooManyRequestsException* as a response. For functions with a long timeout, your client might be disconnected during synchronous invocation while it waits for a response and returns an HTTP 504 error.
The option that says: **There is an authorization failure occurring between API Gateway and the Lambda function** is incorrect because an authentication issue usually produces HTTP 403 errors and not 504s. The gateway response for authorization failures for missing authentication token errors, invalid AWS signature errors, or Amazon Cognito authentication problems is HTTP 403, which is why this option is unlikely to cause this issue.

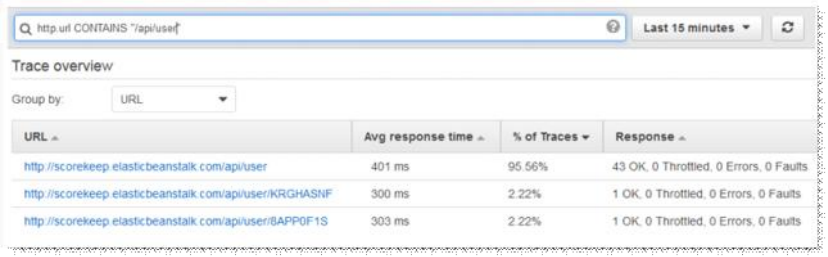
References:
https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html
https://aws.amazon.com/about-aws/whats-new/2017/11/customize-integration-timeouts-in-amazon-api-gateway/
https://docs.aws.amazon.com/apigateway/latest/developerguide/supported-gateway-response-types.html

Check out this Amazon API Gateway Cheat Sheet:
https://tutorialsdojo.com/amazon-api-gateway/

4. **4. Question**
Category: CDA – Troubleshooting and Optimization
A developer uses AWS X-Ray to create a trace on an instrumented web application to identify any performance bottlenecks. The segment documents being sent by the application contain annotations that the developer wants to utilize in order to identify and filter out specific data from the trace.
Which of the following should the developer do in order to satisfy this requirement with minimal configuration? (Select TWO.)

- Fetch the data using the `BatchGetTraces` API.
- Configure Sampling Rules in the AWS X-Ray Console.
- Fetch the trace IDs and annotations using the `GetTraceSummaries` API.
- Use filter expressions via the X-Ray console.
- Send trace results to an S3 bucket then query the trace output using Amazon Athena.

Incorrect
The compute resources running your application logic send data about their work as **segments**. A segment provides the resource's name, details about the request, and details about the work done.
A subset of segment fields are indexed by X-Ray for use with filter expressions. You can search for segments associated with specific information in the X-Ray console or by using the `GetTraceSummaries` API.



Even with sampling, a complex application generates a lot of data. When you choose a time period of traces to view in the X-Ray console, you might get more results than the console can display. You can narrow the results to just the traces that you want to find by using a **filter expression**. Running the `GetTraceSummaries` operation retrieves IDs and annotations for traces available for a specified time frame using an optional filter.
Hence, **using filter expressions via the X-Ray console** and **fetching the trace IDs and annotations using the `GetTraceSummaries` API** are the correct answers in this scenario.
Fetching the data using the `BatchGetTraces` API is incorrect because this API simply retrieves a list of traces specified by ID. It does not support filter expressions nor returns the annotations.
Sending trace results to an S3 bucket then querying the trace output using Amazon Athena is incorrect. Although this solution may work, this entails a lot of configuration which is contrary to what the scenario requires. There are other simpler methods of searching through traces in X-Ray such as using annotations and filter expressions.
Configuring Sampling Rules in the AWS X-Ray Console is incorrect because sampling rules just tell the X-Ray SDK how many requests to record for a set of criteria.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-console-filters.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html
https://docs.aws.amazon.com/xray/latest/api/API_GetTraceSummaries.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

5. **5. Question**
Category: CDA – Troubleshooting and Optimization
A developer is planning to use the AWS Elastic Beanstalk console to run the AWS X-Ray daemon on the EC2 instances in her application environment. She will use X-Ray to construct a service map to help identify issues with her application and to provide insight on which application component to optimize. The environment is using a default Elastic Beanstalk instance profile.
Which IAM managed policy does Elastic Beanstalk use for the X-Ray daemon to upload data to X-Ray?

- `AWSXRayDaemonWriteAccess`
- `AWSXRayElasticBeanstalkWriteAccess`
- `AWSXrayReadOnlyAccess`
- `AWSXrayFullAccess`

Correct
You can use **AWS Identity and Access Management (IAM)** to grant X-Ray permissions to users and compute resources in your account. IAM controls access to the X-Ray service at the API level to enforce permissions uniformly, regardless of which client (console, AWS SDK, AWS CLI) your users employ. To use the X-Ray console to view service maps and segments, you only need read permissions. To enable console access, add the AWSXrayReadOnlyAccess managed policy to your IAM user. For local development and testing, create an IAM user with read and write permissions. Generate access keys for the user and store them in the standard AWS SDK location. You can use these credentials with the X-Ray daemon, the AWS CLI, and the AWS SDK.

To deploy your instrumented app to AWS, create an IAM role with write permissions and assign it to the resources running your application. AWSXRayDaemonWriteAccess includes permission to upload traces, and some read permissions as well to support the use of sampling rules.
The read and write policies do not include permission to configure encryption key settings and sampling rules. Use AWSXrayFullAccess to access these settings, or add configuration APIs in a custom policy. For encryption and decryption with a customer-managed key that you create, you also need permission to use the key.
On supported platforms, you can use a configuration option to run the X-Ray daemon on the instances in your environment. You can enable the daemon in the Elastic Beanstalk console or by using a configuration file. To upload data to X-Ray, the X-Ray daemon requires IAM permissions in the **AWSXRayDaemonWriteAccess** managed policy. These permissions are included in the Elastic Beanstalk instance profile.
Hence, the correct answer is the **AWSXRayDaemonWriteAccess** managed policy.
AWSXrayReadOnlyAccess is incorrect because this policy is primarily used if you just want a read-only access to X-Ray.
AWSXrayFullAccess is incorrect. Although this can provide the required access to the daemon, this is not being used in Elastic Beanstalk as it does not abide by the standard security advice of granting the least privilege.
AWSXRayElasticBeanstalkWriteAccess is incorrect because this is not an available managed policy.

References:
https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environment-configuration-debugging.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-permissions.html
https://docs.aws.amazon.com/xray/latest/devguide/security.html

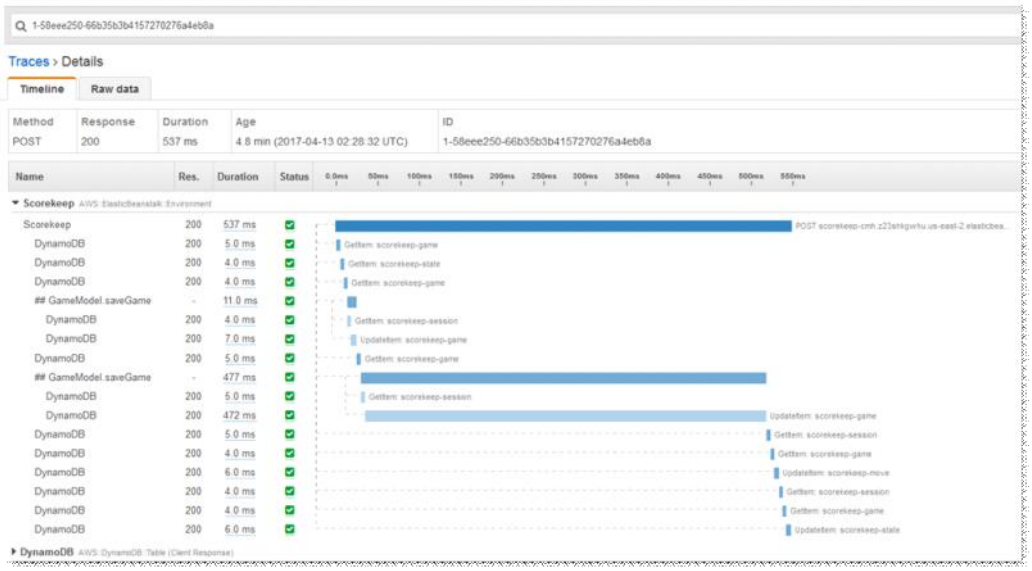
Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

Instrumenting your Application with AWS X-Ray:
https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/

6. **6. Question**
Category: CDA – Troubleshooting and Optimization
A developer has instrumented an application using the X-Ray SDK to collect all data about the requests that an application serves. There is a new requirement to develop a custom debug tool which will enable them to view the full traces of their application without using the X-Ray console.
What should the developer do to accomplish this task?

- Use the `GetServiceGraph` API to get the list of trace IDs of the application and then retrieve the list of traces using `GetTraceSummaries` API.
- Use the `GetTraceSummaries` API to get the list of trace IDs of the application and then retrieve the list of traces using `BatchGetTraces` API.
- Use the `BatchGetTraces` API to get the list of trace IDs of the application and then retrieve the list of traces using `GetTraceSummaries` API.
- Use the `GetGroup` API to get the list of trace IDs of the application and then retrieve the list of traces using `BatchGetTraces` API.

Incorrect
X-Ray compiles and processes segment documents to generate queryable **trace summaries** and **full traces** that you can access by using the `GetTraceSummaries` and `BatchGetTraces` APIs, respectively. In addition to the segments and subsegments that you send to X-Ray, the service uses information in subsegments to generate **inferred segments** and adds them to the full trace. Inferred segments represent downstream services and resources in the service map.



In this scenario, the developer should **use the `GetTraceSummaries` API to get the list of trace IDs of the application and then retrieve the list of traces using `BatchGetTraces` API** in order to develop the custom debug tool
The option that says: **Use the `GetGroup` API to get the list of trace IDs of the application and then retrieving the list of traces using `BatchGetTraces` API** is incorrect because the `GetGroup` API just retrieves the group resource details.
The option that says: **Use the `GetServiceGraph` API to get the list of trace IDs of the application and then retrieving the list of traces using `GetTraceSummaries` API** is incorrect because the `GetServiceGraph` API just shows which services process the incoming requests, including the downstream services that they call as a result. In addition, you have to use the *`BatchGetTraces`* API instead of the *`GetTraceSummaries`* API to retrieve the list of traces.
The option that says: **Use the `BatchGetTraces` API to get the list of trace IDs of the application and then retrieving the list of traces using `GetTraceSummaries` API** is incorrect because it should be the other way around. You have to use the `GetTraceSummaries` API to get the list of trace IDs of the application and then use its result as an input parameter to retrieve the list of traces using the `BatchGetTraces` API.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html
https://docs.aws.amazon.com/xray/latest/api/API_BatchGetTraces.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

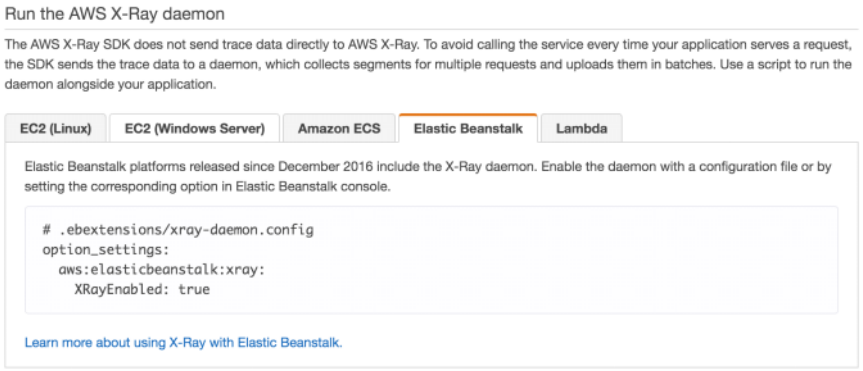
7. **7. Question**
Category: CDA – Troubleshooting and Optimization
A new IT policy requires you to trace all calls that your Node.js application sends to external HTTP web APIs as well as SQL database queries. You have to instrument your application, which is hosted in Elastic Beanstalk, in order to properly trace the calls via the X-Ray console .

What should you do to comply with the given requirement?

- Create a Docker image that runs the X-Ray daemon.
- Enable active tracing in the Elastic Beanstalk by including the `healthcheckurl.config` configuration file in the `.ebextensions` directory of your source code.
- Use a user data script to run the daemon automatically.
- Enable the X-Ray daemon by including the `xray-daemon.config` configuration file in the `.ebextensions` directory of your source code.

Incorrect

You can use the **AWS Elastic Beanstalk** console or a configuration file to run the AWS X-Ray daemon on the instances in your environment. X-Ray is an AWS service that gathers data about the requests that your application serves, and uses it to construct a service map that you can use to identify issues with your application and opportunities for optimization.



To relay trace data from your application to AWS X-Ray, you can run the X-Ray daemon on your Elastic Beanstalk environment's Amazon EC2 instances. Elastic Beanstalk platforms provide a configuration option that you can set to run the daemon automatically. You can enable the daemon in a configuration file in your source code or by choosing an option in the Elastic Beanstalk console. When you enable the configuration option, the daemon is installed on the instance and runs as a service.

Hence, the correct answer is to: **enable the X-Ray daemon by including the `xray-daemon.config` configuration file in the `.ebextensions` directory of your source code**, just as shown above.

Using a user data script to run the daemon automatically is incorrect because this is only applicable if you want to enable X-Ray to your EC2 instances.

Creating a Docker image that runs the X-Ray daemon is incorrect because this is what you need to do if you want to enable X-Ray on ECS Cluster and not on Elastic Beanstalk.

Enabling active tracing in the Elastic Beanstalk by including the `healthcheckurl.config` configuration file in the `.ebextensions` directory of your source code is incorrect because this configuration file only sets the application health check URL and not X-Ray Tracing.

References:

https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-beanstalk.html
https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/environment-configuration-debugging.html

Check out these AWS X-Ray and Elastic Beanstalk Cheat Sheets:

https://tutorialsdojo.com/aws-x-ray/
https://tutorialsdojo.com/aws-elastic-beanstalk/

8. **8. Question**

Category: CDA – Troubleshooting and Optimization

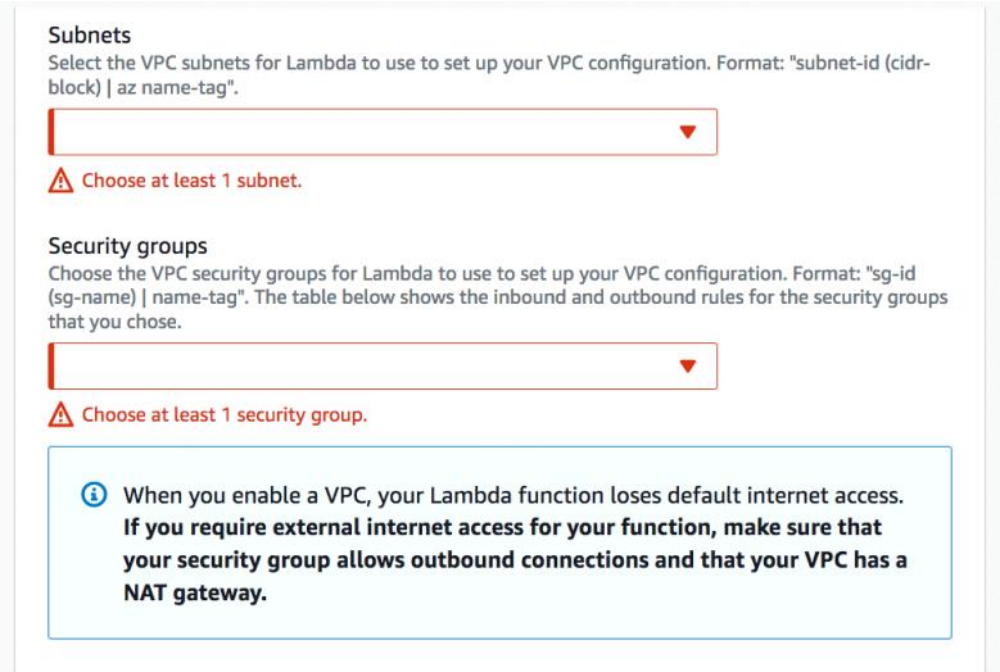
A developer is refactoring a Lambda function that currently processes data using a public GraphQL API. There's a new requirem ent to store query results in a database hosted in a VPC. The function has been configured with additional VPC-specific information, and the database connection has been successfully established. However, the engineer has discovered tha t the function can no longer connect to the internet after testing.

Which of the following should the developer do to fix this issue? (Select TWO.)

- Set up elastic network interfaces (ENIs) to enable your Lambda function to connect securely to other resources within your pr ivate VPC.
- Configure your function to forward payloads that were not processed to a dead-letter queue (DLQ) using Amazon SQS.
- Add a NAT gateway to your VPC.
- Submit a limit increase request to AWS to raise the concurrent executions limit of your Lambda function.
- Ensure that the associated security group of the Lambda function allows outbound connections.

Incorrect

AWS Lambda uses the VPC information you provide to set up ENIs that allow your Lambda function to access VPC resources. Each ENI is assigned a private IP address from the IP address range within the subnets you specify but is not assigned any public IP addresses.



Therefore, if your Lambda function requires Internet access (for example, to access AWS services that don't have VPC endpoints), you can configure a NAT instance inside your VPC, or you can use the Amazon VPC NAT gateway. You cannot use an Internet gateway attached to your VPC since that requires the ENI to have public IP addresses.

If your Lambda function needs Internet access, just as described in this scenario, do not attach it to a public subnet or to a private subnet without Internet access. Instead, attach it only to private subnets with Internet access through a NAT instance or **add a NAT gateway to your VPC**. You should also **ensure that the associated security group of the Lambda function allows outbound connections.**

The option that says: **Submit a limit increase request to AWS to raise the concurrent executions limit of your Lambda function** is incorrect because the root cause of the problem is that the function cannot connect to public GraphQL APIs over the Internet. The scenario doesn't mention anything about a concurrency problem.

The option that says: **Configuring your function to forward payloads that were not processed to a dead-letter queue (DLQ) using Amazon SQS** is incorrect because it will only improve the error handling of your Lambda function. The issue here is the Internet connectivity of your function and not its error handling hence, this option will not solve the problem.

The option that says: **Setting up elastic network interfaces (ENIs) to enable your Lambda function to connect securely to other resources within you r private VPC** is incorrect because this is already done automatically by AWS

Lambda. It uses the VPC information you provide to automatically set up ENIs that allow your Lambda function to access VPC resources. You don't need to do this step in order for your Lambda function to be integrated with your VPC.

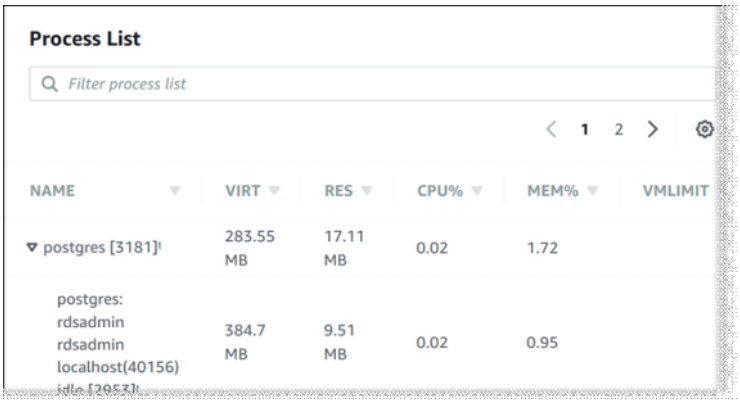
References:
https://docs.aws.amazon.com/lambda/latest/dg/vpc.html
https://aws.amazon.com/premiumsupport/knowledge-center/internet-access-lambda-function/

Check out this AWS Lambda Cheat Sheet:
https://tutorialsdojo.com/aws-lambda/

9. **9. Question**
Category: CDA – Troubleshooting and Optimization
A company has an application hosted in an ECS Cluster that heavily uses an RDS database. A developer needs to closely monitor how the different processes on a DB instance use the CPU, such as the percentage of the CPU bandwidth or the total memory consumed by each process to ensure application performance.
Which of the following is the MOST suitable solution that the developer should implement?

- Track the `CPU%` and `MEM%` metrics which are readily available in the Amazon RDS console.
- Use Enhanced Monitoring in RDS.
- Use CloudWatch to track the CPU Utilization of your database.
- Develop a shell script that collects and publishes custom metrics to CloudWatch which tracks the real-time CPU Utilization of the RDS instance.

Incorrect
Amazon RDS provides metrics in real time for the operating system (OS) that your DB instance runs on. You can view the metrics for your DB instance using the console or consume the Enhanced Monitoring JSON output from CloudWatch Logs in a monitoring system of your choice. By default, Enhanced Monitoring metrics are stored in the CloudWatch Logs for 30 days. To modify the amount of time the metrics are stored in the CloudWatch Logs, change the retention for the `RDSOSMetrics` log group in the CloudWatch console.



Take note that there are certain differences between CloudWatch and Enhanced Monitoring Metrics. CloudWatch gathers metrics about CPU utilization from the hypervisor for a DB instance, and Enhanced Monitoring gathers its metrics from an agent on the instance. As a result, you might find differences between the measurements, because the hypervisor layer performs a small amount of work.
The differences can be greater if your DB instances use smaller instance classes because then there are likely more virtual machines (VMs) that are managed by the hypervisor layer on a single physical instance. Enhanced Monitoring metrics are useful when you want to see how different processes or threads on a DB instance use the CPU.
Hence, the correct answer is to **use Enhanced Monitoring in RDS.**
Developing a shell script that collects and publishes custom metrics to CloudWatch which tracks the real-time CPU Utilization of the RDS instance is incorrect. Although you can use Amazon CloudWatch Logs and CloudWatch dashboard to monitor the CPU Utilization of the database instance, using CloudWatch alone is still not enough to get the specific percentage of the CPU bandwidth and total memory consumed by each database process. The data provided by CloudWatch is not as detailed as compared with the Enhanced Monitoring feature in RDS. Take note as well that you do not have direct access to the instances/servers of your RDS database instance, unlike with your EC2 instances where you can install a CloudWatch agent or a custom script to get CPU and memory utilization of your instance.
Using CloudWatch to track the CPU Utilization of your database is incorrect. Although you can use Amazon CloudWatch to monitor the CPU Utilization of your database instance, it does not provide the percentage of the CPU bandwidth and total memory consumed by each database process in your RDS instance. Take note that CloudWatch gathers metrics about CPU utilization from the hypervisor for a DB instance, while RDS Enhanced Monitoring gathers its metrics from an agent on the instance.
Tracking the `CPU%` and `MEM%` metrics which are readily available in the Amazon RDS console is incorrect because these metrics are not readily available in the Amazon RDS console, which is contrary to what is being stated in this option.

References:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_Monitoring.OS.html#USER_Monitoring.OS.CloudWatchLogs
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/MonitoringOverview.html#monitoring-cloudwatch

Check out these Amazon CloudWatch and RDS Cheat Sheets:
https://tutorialsdojo.com/amazon-cloudwatch/
https://tutorialsdojo.com/amazon-relational-database-service-amazon-rds/

10. **10. Question**
Category: CDA – Troubleshooting and Optimization
A newly hired developer has been instructed to debug an application. She tried to access the X-Ray console to view service maps and segments but her current access is insufficient.
Which of the following is the MOST appropriate managed policy that should be granted to the developer?

- `AWSXrayReadOnlyAccess`
- `AWSXRayDaemonWriteAccess`
- `AmazonS3ReadOnlyAccess`
- `AWSXrayFullAccess`

Incorrect
You can use AWS Identity and Access Management (IAM) to grant X-Ray permissions to users and compute resources in your account. IAM controls access to the X-Ray service at the API level to enforce permissions uniformly, regardless of which client (console, AWS SDK, AWS CLI) your users employ. To use the X-Ray console to view service maps and segments, you only need read permissions. To enable console access, add the **AWSXrayReadOnlyAccess** managed policy to your IAM user.
For local development and testing, create an IAM user with read and write permissions. Generate access keys for the user and store them in the standard AWS SDK location. You can use these credentials with the X-Ray daemon, the AWS CLI, and the AWS SDK.
To deploy your instrumented app to AWS, create an IAM role with write permissions and assign it to the resources running your application. **AWSXRayDaemonWriteAccess** includes permission to upload traces and some read permissions as well to support the use of sampling rules.
The read and write policies do not include permission to configure encryption key settings and sampling rules. Use **AWSXrayFullAccess** to access these settings or add configuration APIs in a custom policy. For encryption and decryption with a customer managed key that you create, you also need permission to use the key.
Hence, the **`AWSXrayReadOnlyAccess`** managed policy is the most appropriate one to grant to the developer in order for her to access the X-Ray console. This also abides with the standard security advice of granting least privilege, or granting only the permissions required to perform a task.
`AWSXRayDaemonWriteAccess` is incorrect because this policy is more suitable if you want to grant permission to upload traces to X-Ray.
`AWSXrayFullAccess` is incorrect. Although this can provide the required access to the developer, it does not abide with the standard security advice of granting least privilege. Hence, this is not the most appropriate policy to use.
`AmazonS3ReadOnlyAccess` is incorrect because this policy just provides the instance permission to download the X-Ray daemon from Amazon S3.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-permissions.html
https://docs.aws.amazon.com/xray/latest/devguide/security.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

11. **11. Question**
Category: CDA – Troubleshooting and Optimization
A Docker application hosted on an ECS cluster has encountered intermittent unavailability issues and timeouts. The lead DevOps engineer instructed you to instrument the application to detect where high latencies are occurring and to determine the specific services and paths impacting application performance.
Which of the following steps should you take to accomplish this task properly? (Select TWO.)

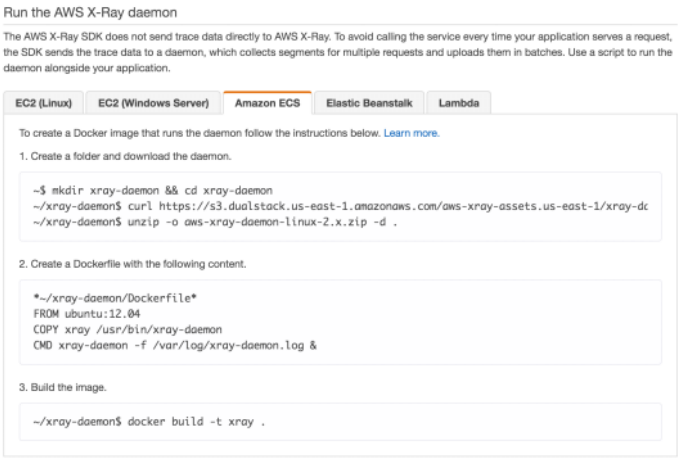
- Add the `xray-daemon.config` configuration file in your Docker image

- Create a Docker image that runs the X-Ray daemon, upload it to a Docker image repository, and then deploy it to your Amazon ECS cluster.
- Configure the port mappings and network mode settings in the container agent to allow traffic on TCP port 2000.
- Configure the port mappings and network mode settings in your task definition file to allow traffic on UDP port 2000.
- Manually install the X-Ray daemon to the instances via a user data script.

Incorrect

The AWS X-Ray SDK does not send trace data directly to AWS X-Ray. To avoid calling the service every time your application serves a request, the SDK sends the trace data to a daemon, which collects segments for multiple requests and uploads them in batches. Use a script to run the daemon alongside your application.

To properly instrument your applications in Amazon ECS, you have to create a Docker image that runs the X-Ray daemon, upload it to a Docker image repository, and then deploy it to your Amazon ECS cluster. You can use port mappings and network mode settings in your task definition file to allow your application to communicate with the daemon container.



The AWS X-Ray daemon is a software application that listens for traffic on UDP port 2000, gathers raw segment data, and relays it to the AWS X-Ray API. The daemon works in conjunction with the AWS X-Ray SDKs and must be running so that data sent by the SDKs can reach the X-Ray service.

Hence, the correct steps to properly instrument the application is to **create a Docker image that runs the X-Ray daemon, upload it to a Docker image repository, and then deploy it to your Amazon ECS cluster.** In addition, **you also have to configure the port mappings and network mode settings in your task definition file to allow traffic on UDP port 2000.**

Configuring the port mappings and network mode settings in the container agent to allow traffic on TCP port 2000 is incorrect because this should be done in the task definition and not in the container agent. Moreover, X-Ray is primarily using the UDP port 2000, so this should also be added alongside with the TCP port mapping.

Manually installing the X-Ray daemon to the instances via a user data script is incorrect because this is only applicable if your application is hosted in an EC2 instance.

Adding the `xray-daemon.config` configuration file in your Docker image is incorrect because this step is not suitable for ECS. The `xray-daemon.config` configuration file is primarily used in Elastic Beanstalk.

References:

https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-ecs.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon.html
https://docs.aws.amazon.com/xray/latest/devguide/scorekeep-ecs.html

Check out this AWS X-Ray Cheat Sheet:

https://tutorialsdojo.com/aws-x-ray/

Instrumenting your Application with AWS X-Ray:

https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/

12. **12. Question**

Category: CDA – Troubleshooting and Optimization

A developer is utilizing AWS X-Ray to generate a visual representation of the requests flowing through their enterprise web application. Since the application interacts with multiple services, all requests must be traced in X-Ray, including any downstream calls made to AWS resources.

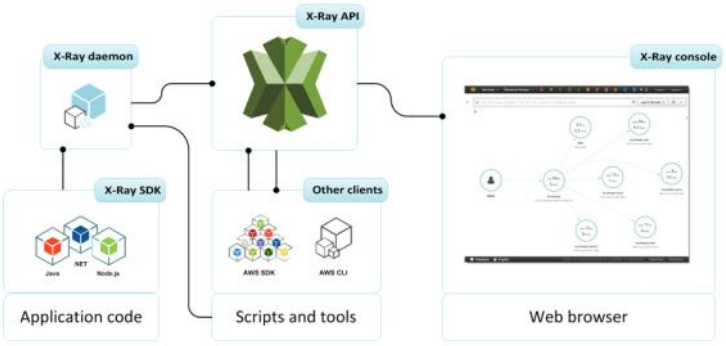
Which of the following actions should the developer implement for this scenario?

- Use AWS X-Ray SDK to upload a trace segment by executing `PutTraceSegments` API.
- Install AWS X-Ray on the different services that communicate with the application including the AWS resources that the application calls.
- Use X-Ray SDK to generate segment documents with subsegments and send them to the X-Ray daemon, which will buffer them and upload to the X-Ray API in batches.
- Pass multiple trace segments as a parameter of `PutTraceSegments` API.

Incorrect

You can send trace data to X-Ray in the form of segment documents. A **segment document** is a JSON formatted string that contains information about the work that your application does in service of a request. Your application can record data about the work that it does itself in segments or work that uses downstream services and resources in subsegments.

A segment document can be up to 64 kB and contain a whole segment with subsegments, a fragment of a segment that indicates that a request is in progress, or a single subsegment that is sent separately. You can send segment documents directly to X-Ray by using the `PutTraceSegments` API. An alternative is, instead of sending segment documents to the X-Ray API, you can send segments and subsegments to an X-Ray daemon, which will buffer them and upload to the X-Ray API in batches. The X-Ray SDK sends segment documents to the daemon to avoid making calls to AWS directly. This is the correct option among the choices.



Hence, **using X-Ray SDK to generate segment documents with subsegments and sending them to the X-Ray daemon, which will buffer them and upload to the X-Ray API in batches** is the correct answer in this scenario.

Using AWS X-Ray SDK to upload a trace segment by executing `PutTraceSegments` API is incorrect because you should upload the segment documents with subsegments instead. A trace segment is just a JSON representation of a request that your application serves.

Installing AWS X-Ray on the different services that communicate with the application including the AWS resources that the application calls is incorrect because you cannot run a trace on the application and the services at the same time as this will produce two different results. You simply have to send the segment documents with subsegments to get the information about downstream calls that your application makes to AWS resources.

Passing multiple trace segments as a parameter of `PutTraceSegments` API is incorrect because, contrary to the API's name, you have to upload **segment** documents and not trace segments. The API has a single parameter: `TraceSegmentDocuments`, that takes a list of JSON segment documents.

References:

https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-api-sendingdata.html
https://docs.aws.amazon.com/xray/latest/api/API_PutTraceSegments.html

Check out this AWS X-Ray Cheat Sheet:

https://tutorialsdojo.com/aws-x-ray/

13. **13. Question**

Category: CDA – Troubleshooting and Optimization

The developer has built a real-time IoT device monitoring application that leverages Amazon Kinesis Data Stream to ingest data. The application uses several EC2 instances for processing. Recently, the developer has observed a steady increase in the rate of data flowing into the stream, indicating that the stream's capacity must be scaled up to sustain optimal performance.

What should the developer do to increase the capacity of the stream?

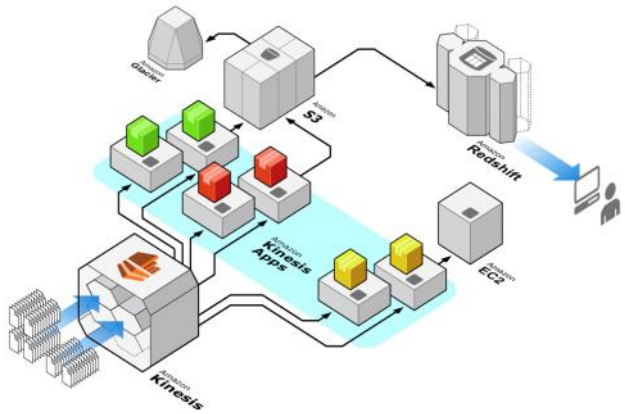
- Integrate Amazon Data Firehose with the Amazon Kinesis Data Stream to increase the capacity of the stream.
- Split every shard in the stream.
- Merge every shard in the stream.
- Upgrade the instance type of the EC2 instances.

Incorrect

Amazon Kinesis Data Streams is a massively scalable, highly durable data ingestion and processing service optimized for streaming data. You can configure hundreds of thousands of data producers to continuously put data into a Kinesis data stream. Data will be available within milliseconds to your Amazon Kinesis applications, and those applications will receive data records in the order they were generated.

The purpose of resharding in Amazon Kinesis Data Streams is to enable your stream to adapt to changes in the rate of data flow. You split shards to increase the capacity (and cost) of your stream. You merge shards to reduce the cost (and capacity) of your stream.

One approach to resharding could be to split every shard in the stream—which would double the stream's capacity. However, this might provide more additional capacity than you actually need and, therefore, create unnecessary costs.



You can also use metrics to determine which are your "hot" or "cold" shards, that is, shards that are receiving much more data or much less data than expected. You could then **selectively split the hot shards to increase capacity** for the hash keys that target those shards. Similarly, you could merge cold shards to make better use of their unused capacity.

You can obtain some performance data for your stream from the Amazon CloudWatch metrics that Kinesis Data Streams publishes. However, you can also collect some of your own metrics for your streams. One approach would be to log the hash key values generated by the partition keys for your data records. Recall that you specify the partition key at the time that you add the record to the stream.

Hence, the correct answer is: **Split every shard in the stream.**

The option that says: **Upgrade the instance type of the EC2 instances** is incorrect. Although it will improve the processing time of the data in the stream, it will not increase the capacity of the stream itself. You have to reshard the stream in order to increase or decrease its capacity, and not just upgrade the EC2 instances which process the data in the stream.

The option that says: **Merge every shard in the stream** is incorrect because merging shards will actually decrease the capacity of the stream rather than increase it. This is only useful if you want to save costs or if the data stream is underutilized, which are both not indicated in the scenario.

The option that says: **Integrate Amazon Data Firehose with the Amazon Kinesis Data Stream to increase the capacity of the stream** is incorrect because Data Firehose just provides a way to reliably transform and load streaming data into data stores and analytics tools. This method will not increase the capacity of the stream as it doesn't mention anything about resharding.

References:

https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding-strategies.html
https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding.html

Check out this Amazon Kinesis Cheat Sheet:

https://tutorialsdojo.com/amazon-kinesis/

Kinesis Scaling, Resharding and Parallel Processing:

https://tutorialsdojo.com/kinesis-scaling-resharding-and-parallel-processing/

14. **14. Question**

Category: CDA – Troubleshooting and Optimization

A leading insurance firm is hosting its customer portal in Elastic Beanstalk, which has an RDS database in AWS. The support team in your company discovered a lot of SQL injection attempts and cross-site scripting attacks on the portal, which is starting to affect the production environment.

Which of the following services should you implement to mitigate this attack?

- AWS WAF
- Network Access Control List
- AWS Firewall Manager
- Amazon GuardDuty

Incorrect

AWS WAF is a web application firewall that lets you monitor the HTTP and HTTPS requests that are forwarded to an Amazon API Gateway API, Amazon CloudFront or an Application Load Balancer. AWS WAF also lets you control access to your content. Based on conditions that you specify, such as the IP addresses that requests originate from or the values of query strings, API Gateway, CloudFront or an Application Load Balancer responds to requests either with the requested content or with an HTTP 403 status code (Forbidden). You also can configure CloudFront to return a custom error page when a request is blocked.

At the simplest level, AWS WAF lets you choose one of the following behaviors:

Allow all requests except the ones that you specify – This is useful when you want CloudFront or an Application Load Balancer to serve content for a public website, but you also want to block requests from attackers.

Block all requests except the ones that you specify – This is useful when you want to serve content for a restricted website whose users are readily identifiable by properties in web requests, such as the IP addresses that they use to browse to the website.

Count the requests that match the properties that you specify – When you want to allow or block requests based on new properties in web requests, you first can configure AWS WAF to count the requests that match those properties without allowing or blocking those requests. This lets you confirm that you didn't accidentally configure AWS WAF to block all the traffic to your website. When you're confident that you specified the correct properties, you can change the behavior to allow or block requests.

Hence, the correct answer in this scenario is **AWS WAF.**

Amazon GuardDuty is incorrect because this is just a threat detection service that continuously monitors malicious activity and unauthorized behavior to protect your AWS accounts and workloads.

AWS Firewall Manager is incorrect because this just simplifies your AWS WAF and AWS Shield Advanced administration and maintenance tasks across multiple accounts and resources.

Network Access Control List is incorrect because this is an optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets.

References:

https://aws.amazon.com/waf/

https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html
https://aws.amazon.com/blogs/security/three-most-important-aws-waf-rate-based-rules/

Check out this AWS WAF Cheat Sheet:
https://tutorialsdojo.com/aws-waf/

15. **15. Question**

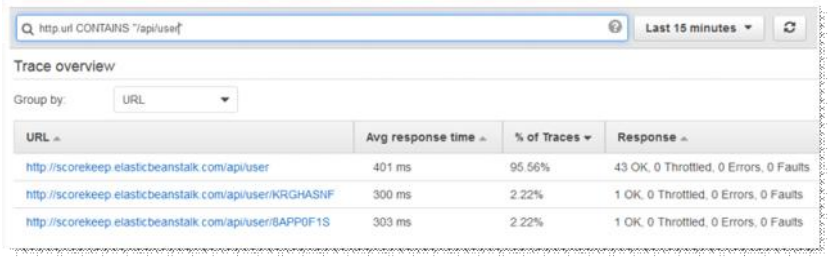
Category: CDA – Troubleshooting and Optimization

An application, which already uses X-Ray, generates thousands of trace data every hour. The developer wants to use a filter expression that will limit the results based on custom attributes or keys that he specifies.
How should the developer refactor the application in order to filter the results in the X-Ray console?

- Create a new sampling rule based on the custom attributes.
- Add the custom attributes as metadata in your segment document.
- Add the custom attributes as annotations in your segment document.
- Include the custom attributes as new segment fields in the segment document.

Incorrect

Even with sampling, a complex application generates a lot of data. The AWS X-Ray console provides an easy-to-navigate view of the service graph. It shows health and performance information that helps you identify issues and opportunities for optimization in your application. For advanced tracing, you can drill down to traces for individual requests or use **filter expressions** to find traces related to specific paths or users.



When you instrument your application, the X-Ray SDK records information about incoming and outgoing requests, the AWS resources used, and the application itself. You can add other information to the segment document as annotations and metadata.
Annotations are simple key-value pairs that are indexed for use with filter expressions. Use annotations to record data that you want to use to group traces in the console, or when calling the `GetTraceSummaries` API. X-Ray indexes up to 50 annotations per trace.
Metadata are key-value pairs with values of any type, including objects and lists, but that are not indexed. Use metadata to record data you want to store in the trace but don't need to use for searching traces. You can view annotations and metadata in the segment or subsegment details in the X-Ray console.
Hence, **adding the custom attributes as annotations in your segment document** is the correct answer.
Including the custom attributes as new segment fields in the segment document is incorrect because a segment field can't be used as a filter expression. You have to add the custom attributes as annotations to the segment document that you'll send to X-Ray, just as mentioned above.
Creating a new sampling rule based on the custom attributes is incorrect because sampling is primarily used to ensure efficient tracing and to provide a representative sample of the requests that your application serves.
Adding the custom attributes as metadata in your segment document is incorrect because metadata is primarily used to record custom data that you want to store in the trace but not for searching traces.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html#xray-concepts-annotations
https://docs.aws.amazon.com/xray/latest/devguide/xray-console-filters.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

16. **16. Question**

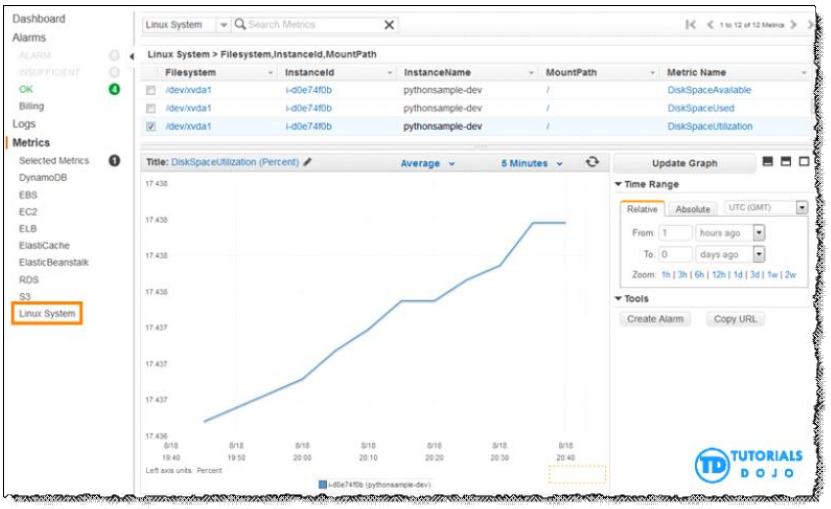
Category: CDA – Troubleshooting and Optimization

A financial company has a cryptocurrency application that has been hosted in Elastic Beanstalk for a couple of months. Recently, the application's performance has been degrading, so you decided to check the CPU and memory utilization of the underlying EC2 instances in CloudWatch. You can see the CPU utilization of the instances but not the memory utilization.
Which of the following is the MOST likely cause of this issue?

- X-Ray Daemon is not installed on the EC2 instances.
- CloudWatch does not track memory utilization by default.
- The detailed monitoring is not enabled in CloudWatch.
- The `.ebextensions/xray-daemon.config` file in Elastic Beanstalk is missing.

Incorrect

Amazon CloudWatch agent enables you to collect both system metrics and log files from Amazon EC2 instances and on-premises servers. The agent supports both Windows Server and Linux and allows you to select the metrics to be collected, including sub-resource metrics such as per-CPU core.



Metrics are data about the performance of your systems. By default, several services provide free metrics for resources (such as Amazon EC2 instances, Amazon EBS volumes, and Amazon RDS DB instances). You can also enable detailed monitoring on some resources, such as your Amazon EC2 instances, or publish your own application metrics. Amazon CloudWatch can load all the metrics in your account (both AWS resource metrics and application metrics that you provide) for search, graphing, and alarms.
However, take note that CloudWatch does not monitor the memory, swap, and disk space utilization of your instances. If you need to track these metrics, you can install a CloudWatch agent in your EC2 instances.
Hence, the correct answer is: **CloudWatch does not track memory utilization by default.**
The option that says: **The detailed monitoring is not enabled in CloudWatch** is incorrect because this will just send metric data for your instance to CloudWatch in 1-minute periods, but not including the memory utilization.
The option that says: **X-Ray Daemon is not installed to the EC2 instances** is incorrect because X-Ray is primarily used for troubleshooting applications and not to monitor the actual EC2 instances. Even if the X-Ray Daemon is installed and is running in the instance, it will still not send the memory utilization metrics to CloudWatch.
The option that says: **The `.ebextensions/xray-daemon.config` file in Elastic Beanstalk is missing** is incorrect because just like what is mentioned above, X-Ray will not send the memory utilization of the EC2 instance.

References:
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Install-CloudWatch-Agent.html
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/viewing_metrics_with_cloudwatch.html
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/monitoring_ec2.html

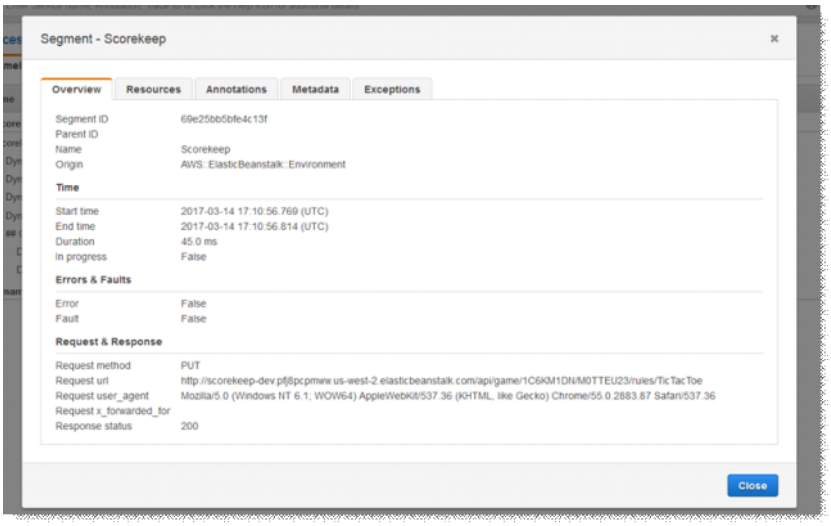
Check out these Amazon EC2 and CloudWatch Cheat Sheets:

https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/
https://tutorialsdojo.com/amazon-cloudwatch/

17. **17. Question**
Category: CDA – Troubleshooting and Optimization
A leading commercial bank has an online banking portal that is hosted in an Auto Scaling group of EC2 instances with an Appli cation Load Balancer in front to distribute the incoming traffic. The application has been instrumented, and the X-Ray daemon has been installed in all instances to allow debugging and troubleshooting using AWS X -Ray.
In this architecture, from which source will AWS X-Ray fetch the client IP address?

- From the `X-Forwarded-For` header of the request.
- From the `ipAddress` query parameter of the request if it exists.
- From the `X-Forwarded-Host` header of the request.
- From the source IP of the IP packet.

Incorrect
AWS X-Ray receives data from services as *segments*. X-Ray then groups segments that have a common request into *traces*. X-Ray processes the traces to generate a *service graph* that provides a visual representation of your application.
The compute resources running your application logic send data about their work as **segments**. A segment provides the resource's name, details about the request, and details about the work done. For example, when an HTTP request reaches your application, it can record the following data about:
The host – hostname, alias or IP address
The request – method, client address, path, user agent
The response – status, content
The work done – start and end times, subsegments
Issues that occur – errors, faults and exceptions, including automatic capture of exception stacks.



The X-Ray SDK gathers information from request and response headers, the code in your application, and metadata about the AWS resources on which it runs. You choose the data to collect by modifying your application configuration or code to instrument incoming requests, downstream requests, and AWS SDK clients.
If a load balancer or other intermediary forwards a request to your application, X-Ray takes the client IP from the `X-Forwarded-For` header in the request instead of from the source IP in the IP packet. The client IP that is recorded for a forwarded request can be forged, so it should not be trusted.
Hence, the correct answer in this scenario is that, AWS X-Ray will fetch the client IP address **from the `X-Forwarded-For` header of the request**.
The option that says: **from the `X-Forwarded-Host` header of the *request*** is incorrect because this header is primarily used in identifying the original host where the request originated from and not the IP address.
The option that says: **from the `ipAddress` query parameter of the request if it exists** is incorrect because query parameters are primarily used in the application layer and not for the network layer. Take note that AWS X-Ray uses the *`X-Forwarded-For`* header of the request and not any query parameter.
The option that says: **from the source IP of the IP packet** is incorrect because AWS X-Ray uses the *`X-Forwarded-For`* header of the request and not the source IP of the IP packet since the application is using an Application Load Balancer.

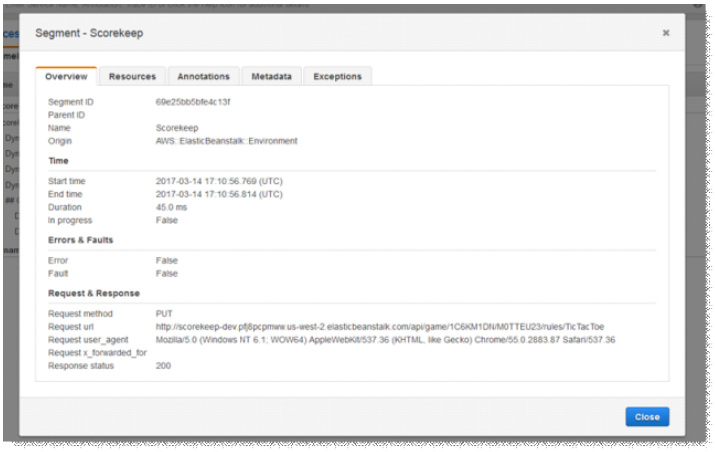
References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

18. **18. Question**
Category: CDA – Troubleshooting and Optimization
A developer is instrumenting an application that will be hosted in a large On-Demand EC2 instance in AWS. All of the downstream calls invoked by the application must be traced properly, including the AWS SDK calls. A user-defined data should also be present to expedite the troubleshooting process.
Which of the following are valid considerations in AWS X-Ray that the developer should follow? (Select TWO.)

- Set the `namespace` subsegment field to `aws` for AWS SDK calls and `remote` for other downstream calls.
- Set the `annotations` object with any additional custom data that you want to store in the segment.
- Set the `namespace` subsegment field to `remote` for AWS SDK calls and `aws` for other downstream calls.
- Set the `metadata` object with any additional custom data that you want to store in the segment.
- Set the `metadata` object with key-value pairs that you want X-Ray to index for search.

Incorrect
A segment document conveys information about a segment to X-Ray. A segment document can be up to 64 kB and contain a whole segment with subsegments, a fragment of a segment that indicates that a request is in progress, or a single subsegment that is sent separately. You can send segment documents directly to X-Ray by using the `PutTraceSegments` API.
X-Ray compiles and processes segment documents to generate queryable trace summaries and full traces that you can access by using the `GetTraceSummaries` and `BatchGetTraces` APIs, respectively. In addition to the segments and subsegments that you send to X-Ray, the service uses information in subsegments to generate inferred segments and adds them to the full trace. Inferred segments represent downstream services and resources in the service map.



A subset of segment fields is indexed by X-Ray for use with filter expressions. For example, if you set the `user` field on a segment to a unique identifier, you can search for segments associated with specific users in the X-Ray console or by using the `GetTraceSummaries` API.

Below are the optional subsegment fields:
`namespace` – `aws` for AWS SDK calls; `remote` for other downstream calls.
`http` – http object with information about an outgoing HTTP call.
`aws` – aws object with information about the downstream AWS resource that your application called.
`error`, `throttle`, `fault`, and `cause` – error fields that indicate an error occurred and that include information about the exception that caused the error.
`annotations` – annotations object with key-value pairs that you want X-Ray to index for search.
`metadata` – metadata object with any additional data that you want to store in the segment.
`subsegments` – array of subsegment objects.
`precursor_ids` – array of subsegment IDs that identify subsegments with the same parent that was completed prior to this subsegment.
You can use the "metadata" field in the segment section to add custom data for your tracing. If you want to trace all the AWS SDK calls of your application, then you can add a subsegment and set the "namespace" field to "AWS". Alternatively, you can set the "namespace" value to "remote" if you want to trace other downstream calls.
Hence, the valid considerations in this scenario are:
– Set the `namespace` subsegment field to `aws` for AWS SDK calls and `remote` for other downstream calls.
– Set the `metadata` object with any additional custom data that you want to store in the segment.
Setting the `annotations` object with any additional custom data that you want to store in the segment is incorrect because this should be the `metadata` object and not the `annotations` object. Segments and subsegments can include an annotations object containing one or more fields that X-Ray indexes for use with filter expressions. Fields can have string, number, or Boolean values (no objects or arrays). X-Ray indexes up to 50 annotations per trace.
Setting the `namespace` subsegment field to `remote` for AWS SDK calls and `aws` for other downstream calls is incorrect because this should be the other way around. You should set the `namespace` subsegment field to `aws` for AWS SDK calls and `remote` for other downstream calls
Setting the `metadata` object with key-value pairs that you want X-Ray to index for search is incorrect because this should be the `annotations` object, and not the `metadata` object.

References:
https://docs.aws.amazon.com/xray/latest/devguide/xray-api-segmentdocuments.html
https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html

Check out this AWS X-Ray Cheat Sheet:

https://tutorialsdojo.com/aws-x-ray/

Instrumenting your Application with AWS X-Ray:
https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/

19. **19. Question**

Category: CDA – Troubleshooting and Optimization

A leading financial company has recently deployed its application to AWS using Lambda and API Gateway. However, they noticed that all metrics are being populated in their CloudWatch dashboard except for `CacheHitCount` and `CacheMissCount`.

What could be the MOST likely cause of this issue?

- They have not provided an IAM role to their API Gateway yet.
- API Gateway Private Integrations has not been configured yet.
- The provided IAM role to their API Gateway only has read access but no write privileges to CloudWatch.
- API Caching is not enabled in API Gateway.

Incorrect

You can monitor API execution using CloudWatch, which collects and processes raw data from API Gateway into readable, near-real-time metrics. These statistics are recorded for a period of two weeks so that you can access historical information and gain a better perspective on how your web application or service is performing. By default, API Gateway metric data is automatically sent to CloudWatch in one-minute periods.

The metrics reported by API Gateway provide information that you can analyze in different ways. The list below shows some common uses for the metrics. These are suggestions to get you started, not a comprehensive list.
– Monitor the **IntegrationLatency** metrics to measure the responsiveness of the backend.
– Monitor the **Latency** metrics to measure the overall responsiveness of your API calls.
– Monitor the **CacheHitCount** and **CacheMissCount** metrics to optimize cache capacities to achieve a desired performance. CacheMissCount tracks the number of requests served from the backend in a given period, **when API caching is enabled**. On the other hand, CacheHitCount track the number of requests served from the API cache in a given period.
Hence, the root cause of this issue is that the **API Caching is not enabled in API Gateway** which is why the ***CacheHitCount*** and ***CacheMissCount*** metrics are not populated.
The option that says: **they have not provided an IAM role to their API Gateway yet** is incorrect because, in the first place, the scenario already mentioned that all metrics are being populated in their CloudWatch dashboard except for two metrics. This implies that some of the metrics are populated which means that the API Gateway already has an IAM Role associated with it.
The option that says: **the provided IAM role to their API Gateway only has read access but no write privileges to CloudWatch** is incorrect because just as what is mentioned above, there is no issue with the IAM Role since all metrics are being populated except only for `CacheHitCount` and `CacheMissCount`. This means that the associated IAM Role already has *write* privileges to write logs to CloudWatch to begin with. The only reason why those two metrics are not being populated is that the API Caching is not enabled.
The option that says: **API Gateway Private Integrations has not been configured yet** is incorrect because this feature only makes it easier to expose your HTTP/HTTPS resources behind an Amazon VPC for access by clients outside of the VPC.

References:
https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-metrics-and-dimensions.html
https://docs.aws.amazon.com/apigateway/latest/developerguide/monitoring-cloudwatch.html

Check out this Amazon API Gateway Cheat Sheet:
https://tutorialsdojo.com/amazon-api-gateway

20. **20. Question**

Category: CDA – Troubleshooting and Optimization

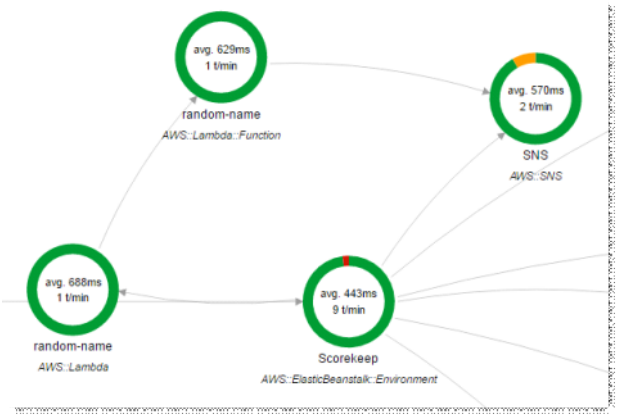
A recently deployed Lambda function has an intermittent issue in processing customer data. You enabled the active tracing option in order to detect, analyze, and optimize performance issues of your function using the X-Ray service.

Which of the following environment variables are used by AWS Lambda to facilitate communication with X-Ray? (Select TWO.)

- `AWS_XRAY_TRACING_NAME`
- `AWS_XRAY_CONTEXT_MISSING`
- `AUTO_INSTRUMENT`
- `_X_AMZN_TRACE_ID`
- `AWS_XRAY_DEBUG_MODE`

Incorrect

AWS X-Ray is an AWS service that allows you to detect, analyze, and optimize performance issues with your AWS Lambda applications. X-Ray collects metadata from the Lambda service and any upstream or downstream services that make up your application. X-Ray uses this metadata to generate a detailed service graph that illustrates performance bottlenecks, latency spikes, and other issues that impact the performance of your Lambda application.



AWS Lambda uses environment variables to facilitate communication with the X-Ray daemon and configure the X-Ray SDK.
_X_AMZN_TRACE_ID: Contains the tracing header, which includes the sampling decision, trace ID, and parent segment ID. If Lambda receives a tracing header when your function is invoked, that header will be used to populate the _X_AMZN_TRACE_ID environment variable. If a tracing header was not received, Lambda will generate one for you.
AWS_XRAY_CONTEXT_MISSING: The X-Ray SDK uses this variable to determine its behavior in the event that your function tries to record X-Ray data, but a tracing header is not available. Lambda sets this value to `LOG_ERROR` by default.
AWS_XRAY_DAEMON_ADDRESS: This environment variable exposes the X-Ray daemon's address in the following format: *`IP_ADDRESS:PORT`*. You can use the X-Ray daemon's address to send trace data to the X-Ray daemon directly without using the X-Ray SDK.
Therefore, the correct answers for this scenario are the **`_X_AMZN_TRACE_ID`** and **`AWS_XRAY_CONTEXT_MISSING`** environment variables.
`AWS_XRAY_TRACING_NAME` is incorrect because this is primarily used in X-Ray SDK where you can set a service name that the SDK uses for segments.
`AWS_XRAY_DEBUG_MODE` is incorrect because this is used to configure the SDK to output logs to the console without using a logging library.
`AUTO_INSTRUMENT` is incorrect because this is primarily used in X-Ray SDK for Django Framework only. This allows the recording of subsegments for built-in database and template rendering operations.

References:
https://docs.aws.amazon.com/lambda/latest/dg/lambda-x-ray.html#viewing-lambda-xray-results
https://docs.aws.amazon.com/xray/latest/devguide/xray-sdk-nodejs-configuration.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-sdk-python-configuration.html

Check out this AWS X-Ray Cheat Sheet:
https://tutorialsdojo.com/aws-x-ray/

Instrumenting your Application with AWS X-Ray:
https://tutorialsdojo.com/instrumenting-your-application-with-aws-x-ray/

21. **21. Question**

Category: CDA – Troubleshooting and Optimization

A serverless application consisting of Lambda functions integrated with API Gateway, and DynamoDB processes ad hoc requests t hat its users send. Due to the recent spike in incoming traffic, some of your customers are complaining that they are getting HTTP 504 errors from time to time.

Which of the following is the MOST likely cause of this issue?

- Since the incoming requests are increasing, the API Gateway automatically enabled throttling which caused the HTTP 504 errors.
- An authorization failure occurred between API Gateway and the Lambda function.
- The usage plan quota has been exceeded for the Lambda function.
- API Gateway request has timed out because the underlying Lambda function has been running for more than 29 seconds.

Incorrect

A gateway response is identified by a response type defined by API Gateway. The response consists of an HTTP status code, a set of additional headers that are specified by parameter mappings, and a payload that is generated by a non-VTL (Apache Velocity Template Language) mapping template.

You can set up a gateway response for a supported response type at the API level. Whenever API Gateway returns a response of the type, the header mappings and payload mapping templates defined in the gateway response are applied to return the mapped results to the API caller.

The following are the Gateway response types which are associated with the HTTP 504 error in API Gateway:

INTEGRATION_FAILURE – The gateway response for an integration failed error. If the response type is unspecified, this response defaults to the DEFAULT_5XX type.

INTEGRATION_TIMEOUT – The gateway response for an integration timed out error. If the response type is unspecified, this response defaults to the DEFAULT_5XX type.

For the integration timeout, the range is from 50 milliseconds to 29 seconds for all integration types, including Lambda, Lambda proxy, HTTP, HTTP proxy, and AWS integrations.

In this scenario, there is an issue where the users are getting HTTP 504 errors in the serverless application. This means the Lambda function is working fine at times but there are instances when it throws an error. Based on this analysis, the most likely cause of the issue is the **INTEGRATION_TIMEOUT** error since you will only get an **INTEGRATION_FAILURE** error if your AWS Lambda integration does not work at all in the first place.

Hence, the root cause of this issue is that the **API Gateway request has timed out because the underlying Lambda function has been running for more than 29 seconds**.

The option that says: **Since the incoming requests are increasing, the API Gateway automatically enabled throttling which caused the HTTP 504 errors** is incorrect because a large number of incoming requests will most likely produce an HTTP 502 or 429 error but not a 504 error. If executing the function would cause you to exceed a concurrency limit at either the account level (***ConcurrentInvocationLimitExceeded***) or function level (***ReservedFunctionConcurrentInvocationLimitExceeded***), Lambda may return a ***TooManyRequestsException*** as a response. For functions with a long timeout, your client might be disconnected during synchronous invocation while it waits for a response and returns an HTTP 504 error.

The option that says: **An authorization failure occurred between API Gateway and the Lambda function** is incorrect because an authentication issue usually produces HTTP 403 errors and not 504s. The gateway response for authorization failures for missing authentication token error, invalid AWS signature error, or Amazon Cognito authentication problems is HTTP 403, which is why this option is unlikely to be the cause of this issue.

The option that says: **The usage plan quota has been exceeded for the Lambda function** is incorrect. Although this is a possible root cause for this scenario, this option has the least chance to produce HTTP 504 errors. The scenario says that the issue happens from *time to time* and not *all the time* which suggests that this happens intermittently. If the usage plan indeed exceeded the quota, then the 504 error should always show up and not just from *time to time*.

References:
https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html
https://aws.amazon.com/about-aws/whats-new/2017/11/customize-integration-timeouts-in-amazon-api-gateway/
https://docs.aws.amazon.com/apigateway/latest/developerguide/supported-gateway-response-types.html

Check out this API Gateway Cheat Sheet:
https://tutorialsdojo.com/amazon-api-gateway/

22. **22. Question**

Category: CDA – Troubleshooting and Optimization

A company is developing an online system that lets patients schedule appointments with their preferred doctors at medical cen ters all over the country. The company uses Amazon DynamoDB as its primary database. The DynamoDB Streams feature is enabled on the DynamoDB table to capture all changes made to the booking data. A Lambda function integrate d with Amazon EventBridge (Amazon CloudWatch Events) is used to process the data stream every 36 hours and then store the results in an S3 bucket.

There are a lot of updated items in DynamoDB that are not sent to the S3 bucket, even though there are no errors in the logs.

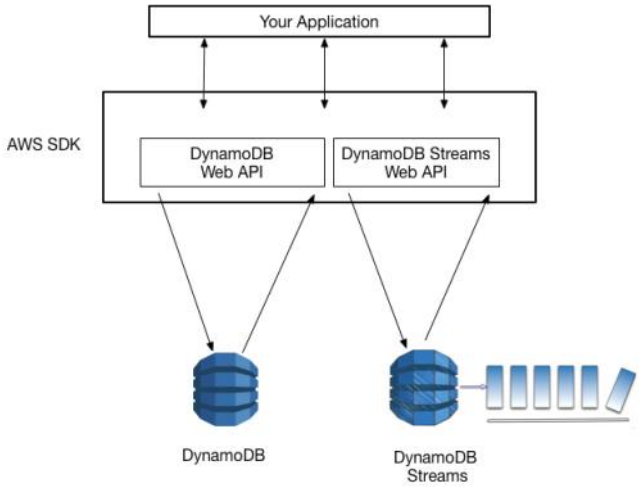
Which of the following is the MOST appropriate solution for this issue?

- Set the value of `StreamViewType` parameter in DynamoDB Streams to `NEW_AND_OLD_IMAGES`.
- Set the value of `StreamViewType` parameter in DynamoDB Streams to `NEW_IMAGE`.
- Decrease the interval of running your function to 24 hours.
- Increase the interval of running your function to 48 hours.

Incorrect

A *DynamoDB stream* is an ordered flow of information about changes to items in an Amazon DynamoDB table. When you enable a stream on a table, DynamoDB captures information about every modification to data items in the table.

Whenever an application creates, updates, or deletes items in the table, DynamoDB Streams writes a stream record with the primary key attribute(s) of the items that were modified. A *stream record* contains information about a data modification to a single item in a DynamoDB table. You can configure the stream so that the stream records capture additional information, such as the "before" and "after" images of modified items.



All data in DynamoDB Streams is subject to a 24-hour lifetime. You can retrieve and analyze the last 24 hours of activity for any given table; however, data older than 24 hours is susceptible to trimming (removal) at any moment. If you disable a stream on a table, the data in the stream will continue to be readable for 24 hours. After this time, the data expires and the stream records are automatically deleted. Note that there is no mechanism for manually deleting an existing stream; you just need to wait until the retention limit expires (24 hours), and all the stream records will be deleted.
Hence, the correct answer is to **decrease the interval of running your function to 24 hours** because in DynamoDB Streams, data older than 24 hours is susceptible to trimming (removal) at any moment.
Increasing the interval of running your function to 48 hours is incorrect because this will actually make the problem worse. Considering that the data in DynamoDB Streams only lasts for 24 hours, you should actually decrease the interval of running your function and not further increase it.
Setting the value of `StreamViewType` parameter in DynamoDB Streams to `NEW_AND_OLD_IMAGES` is incorrect because this just configures DynamoDB to write both the new and the old item images of the item to the stream. Setting the Stream View Type is actually irrelevant in this scenario since the problem is about missing data and not missing old or new values of the items.
Setting the value of `StreamViewType` parameter in DynamoDB Streams to `NEW_IMAGE` is incorrect because this just configures the DynamoDB to write only the new image of the item to the stream. Just as mentioned above, the root cause of the issue is the length of the interval of running the Lambda function and not the DynamoDB Streams configuration.

References:
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html
https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_StreamSpecification.html

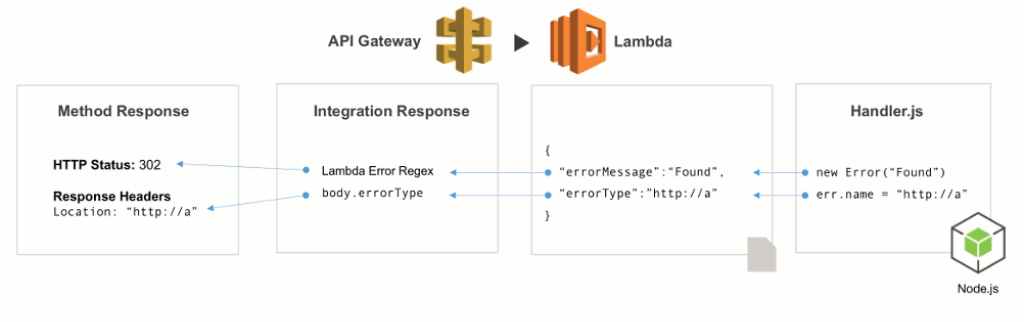
Check out this Amazon DynamoDB Cheat Sheet:
https://tutorialsdojo.com/amazon-dynamodb/

AWS Lambda Integration with Amazon DynamoDB Streams:
https://tutorialsdojo.com/aws-lambda-integration-with-amazon-dynamodb-streams/

23. **23. Question**
Category: CDA – Troubleshooting and Optimization
A developer configured an Amazon API Gateway proxy integration named `MyAPI` to work with a Lambda function. However, when the API is being called, the developer receives a `502 Bad Gateway` error. She tried invoking the underlying function, but it properly returned the result in XML format.
What is the MOST likely root cause of this issue?

- There is an incompatible output returned from a Lambda proxy integration backend.
- There has been an occasional out-of-order invocation due to heavy loads.
- The endpoint request timed-out.
- The API name of the Amazon API Gateway proxy is invalid.

Incorrect
Amazon API Gateway Lambda proxy integration is a simple, powerful, and nimble mechanism to build an API with a setup of a single API method. The Lambda proxy integration allows the client to call a single Lambda function in the backend. The function accesses many resources or features of other AWS services, including calling other Lambda functions



In Lambda proxy integration, when a client submits an API request, API Gateway passes the raw request as-is to the integrated Lambda function, except that the order of the request parameters is not preserved. This request data includes the request headers, query string parameters, URL path variables, payload, and API configuration data. The configuration data can include current deployment stage name, stage variables, user identity, or authorization context (if any). The backend Lambda function parses the incoming request data to determine the response that it returns.
For API Gateway to pass the Lambda output as the API response to the client, the Lambda function must return the result in the following JSON format:

```
{
    "isBase64Encoded": true|false,
    "statusCode": httpStatusCode,
    "headers": { "headerName": "headerValue", ... },
    "body": "..."
}
```

Since the Lambda function returns the result in XML format, it will cause the 502 errors in the API Gateway. Hence, the correct answer is that **there is an incompatible output returned from a Lambda proxy integration backend.**
The option that says: **The API name of the Amazon API Gateway proxy** is invalid is incorrect because there is nothing wrong with its *MyAPI* name.
The option that says: **There has been an occasional out-of-order invocation due to heavy loads** is incorrect. Although this is a valid cause of a 502 error, the issue is most likely caused by the Lambda function's XML response instead of JSON.
The option that says: **The endpoint request timed-out** is incorrect because this will likely result in 504 errors and not 502's.

References:
https://aws.amazon.com/premiumsupport/knowledge-center/malformed-502-api-gateway/
https://docs.aws.amazon.com/apigateway/latest/developerguide/set-up-lambda-proxy-integrations.html#api-gateway-simple-proxy-for-lambda-output-format
https://docs.aws.amazon.com/apigateway/api-reference/handling-errors/

Check out this Amazon API Gateway Cheat Sheet:
https://tutorialsdojo.com/amazon-api-gateway/

24. **24. Question**
Category: CDA – Troubleshooting and Optimization
A web application hosted in Elastic Beanstalk has a configuration file named `.ebextensions/debugging.config` which has the following content:
`option_settings:`

```
aws:elasticbeanstalk:xray:
XRayEnabled: true
```

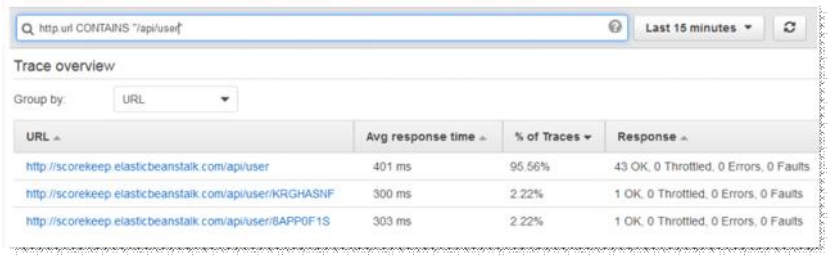
For its database tier, it uses RDS with Multi-AZ deployments configuration and Read Replicas. There is a new requirement to record calls that your application makes to RDS and other internal or external HTTP web APIs. The tracing information should also include the actual SQL database queries sent by the application, which can be searched using the filter expressions in the X-Ray Console.

Which of the following should you do to satisfy the above task?

- Add metadata in the segment document.
- Add annotations in the subsegment section of the segment document.
- Add annotations in the segment document.
- Add metadata in the subsegment section of the segment document.

Incorrect

Even with sampling, a complex application generates a lot of data. The AWS X-Ray console provides an easy-to-navigate view of the service graph. It shows health and performance information that helps you identify issues and opportunities for optimization in your application. For advanced tracing, you can drill down to traces for individual requests, or use **filter expressions** to find traces related to specific paths or users.



When you instrument your application, the X-Ray SDK records information about incoming and outgoing requests, the AWS resources used, and the application itself. You can add other information to the segment document as annotations and metadata.

Annotations are simple key-value pairs that are indexed for use with filter expressions. Use annotations to record data that you want to use to group traces in the console or when calling the `GetTraceSummaries` API. X-Ray indexes up to 50 annotations per trace.

Metadata are key-value pairs with values of any type, including objects and lists, but that is not indexed. Use metadata to record data you want to store in the trace but don't need to use for searching traces. You can view annotations and metadata in the segment or subsegment details in the X-Ray console.

A trace segment is a JSON representation of a request that your application serves. A trace segment records information about the original request, information about the work that your application does locally, and subsegments with information about downstream calls that your application makes to AWS resources, HTTP APIs, and SQL databases.

Hence, **adding annotations in the subsegment section of the segment document** is the correct answer.

Adding annotations in the segment document is incorrect. Although the use of annotations is correct, you have to add this in the ***subsegment*** section of the *segment* document since you want to trace the downstream call to RDS and not the actual request to your application.

Adding metadata in the segment document is incorrect because metadata is primarily used to record custom data that you want to store in the trace but not for searching traces since this can't be picked up by filter expressions in the X-Ray Console. You have to use annotations instead. In addition, you have to add this in the *subsegment* section of the *segment* document since you want to trace the downstream call to RDS and not the actual request to your application.

Adding metadata in the subsegment section of the segment document is incorrect because, just as mentioned above, metadata is just used to record custom data that you want to store in the trace but not for searching traces.

References:

https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html#xray-concepts-annotations
https://docs.aws.amazon.com/xray/latest/devguide/xray-console-filters.html

Check out this AWS X-Ray Cheat Sheet:

https://tutorialsdojo.com/aws-x-ray/

25. **25. Question**

Category: CDA – Troubleshooting and Optimization

You were recently hired as a developer for a leading insurance firm in Asia which has a hybrid cloud architecture with AWS. The project that was assigned to you involves setting up a static website using Amazon S3 with a CORS configuration as shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<CORSConfiguration xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
    <CORSRule>
        <AllowedOrigin>https://tutorialsdojo.com</AllowedOrigin>
        <AllowedMethod>GET</AllowedMethod>
        <AllowedMethod>PUT</AllowedMethod>
        <AllowedMethod>POST</AllowedMethod>
        <AllowedMethod>DELETE</AllowedMethod>
        <AllowedHeader>*</AllowedHeader>
        <ExposeHeader>ETag</ExposeHeader>
        <ExposeHeader>x-amz-meta-custom-header</ExposeHeader>
        <MaxAgeSeconds>3600</MaxAgeSeconds>
    </CORSRule>
</CORSConfiguration>
```

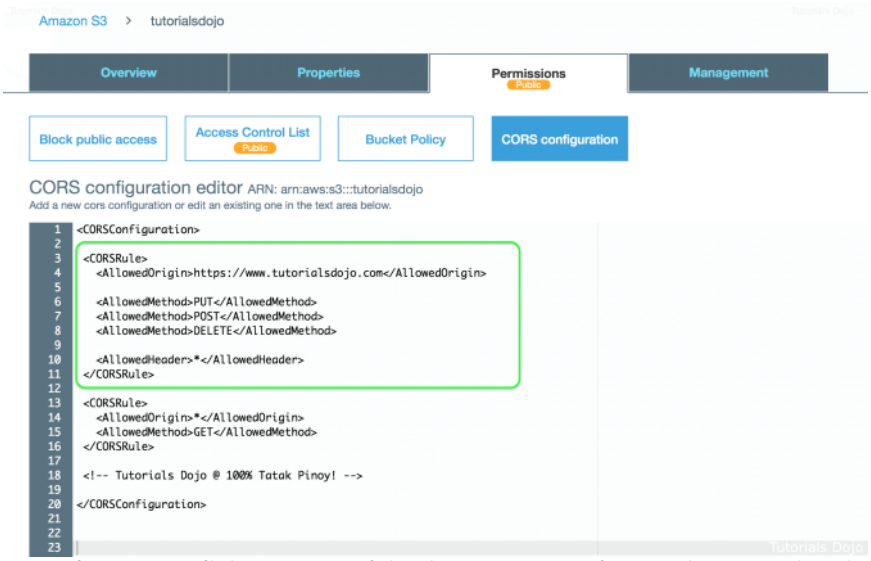
Which of the following statements are TRUE with regards to this S3 configuration? (Select TWO.)

- The request will fail if the `x-amz-meta-custom-header` header is not included.
- This configuration authorizes the user to perform actions on the S3 bucket.
- It allows a user to view, add, remove or update objects inside the S3 bucket from the domain tutorialsdojo.com.
- This will cause the browser to cache the response of the preflight OPTIONS request for 1 hour.
- All HTTP Methods are allowed.

Incorrect

Cross-origin resource sharing (CORS) defines a way for client web applications that are loaded in one domain to interact with resources in a different domain. With CORS support, you can build rich client-side web applications with Amazon S3 and selectively allow cross-origin access to your Amazon S3 resources.

To configure your bucket to allow cross-origin requests, you create a CORS configuration, which is an XML document with rules that identify the origins that you will allow to access your bucket, the operations (HTTP methods) that will support each origin, and other operation-specific information. You can add up to 100 rules to the configuration. You add the XML document as the cors subresource to the bucket either programmatically or by using the Amazon S3 console as shown below:



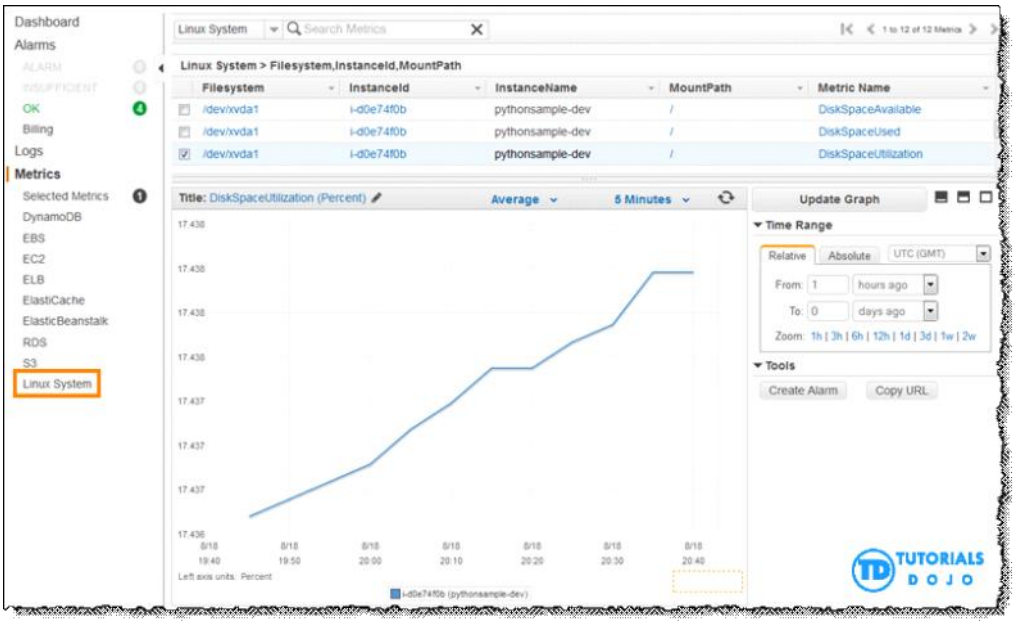
A CORS configuration is an XML file that contains a series of rules within a `<CORSRule>`. A configuration can have up to 100 rules. A rule is defined by one of the following tags:
`AllowedOrigin` – Specifies domain origins that you allow to make cross-domain requests.
`AllowedMethod` – Specifies a type of request you allow (GET, PUT, POST, DELETE, HEAD) in cross-domain requests.
`AllowedHeader` – Specifies the headers allowed in a preflight request.
Below are some of the `CORSRule` elements:
`MaxAgeSeconds` – Specifies the amount of time in seconds (in this example, 3000) that the browser caches an Amazon S3 response to a preflight OPTIONS request for the specified resource. By caching the response, the browser does not have to send preflight requests to Amazon S3 if the original request will be repeated.
`ExposeHeader` – Identifies the response headers (in this example, `x-amz-server-side-encryption`, `x-amz-request-id`, and `x-amz-id-2`) that customers are able to access from their applications (for example, from a JavaScript `XMLHttpRequest` object).
Hence, the correct answers in this scenario are:
– It allows a user to view, add, remove or update objects inside the S3 bucket from the domain tutorialsdojo.com
– This will cause the browser to cache an Amazon S3 response of a preflight OPTIONS request for 1 hour
The option that says: **the request will fail if the `x-amz-meta-custom-header` header is not included** is incorrect because the `ExposeHeader` element refers to the header that will be exposed to the response and not a constraint for the request.
The option that says: **this configuration authorizes the user to perform actions on the S3 bucket** is incorrect because this configuration actually does the opposite. It doesn't authorize the user to perform actions on the S3 bucket.
The option that says: **all HTTP Methods are allowed** is incorrect because the configuration didn't include the `HEAD` HTTP method.

References:
http://docs.aws.amazon.com/AmazonS3/latest/dev/cors.html
https://docs.aws.amazon.com/sdk-for-javascript/v2/developer-guide/cors.html

26. **26. Question**
Category: CDA – Troubleshooting and Optimization
A web application is currently deployed on an On-Demand Linux EC2 instance that connects to an Amazon RDS database. Users have frequently reported that the application crashes intermittently. The support team has reviewed the logs in CloudWatch but has been unable to identify the root cause. To enhance troubleshooting, the team needs to monitor additional metrics, including memory utilization, swap usage, and the count of idle and active processes running on the instance.
Which solution would be the MOST appropriate to implement in this situation?

- Install the AWS X-Ray daemon on the EC2 instance.
- Use detailed monitoring in CloudWatch.
- Use AWS CloudShell to consolidate all metrics in a single dashboard.
- Install the Amazon CloudWatch Logs agent to the EC2 instance.

Incorrect
You can use the CloudWatch agent to collect both system metrics and log files from Amazon EC2 instances and on-premises servers. The agent supports both Windows Server and Linux, and enables you to select the metrics to be collected, including sub-resource metrics such as per-CPU core. Aside from the usual metrics, it also tracks the memory, swap, and disk space utilization metrics of your server.



Hence, the most suitable solution to use in this scenario is to: **Install the Amazon CloudWatch Logs agent to the EC2 instance.**
The option that says: **Use AWS CloudShell to consolidate all metrics in a single dashboard** is incorrect because this service is simply a command-line interface used for managing AWS resources from a terminal. It is not a monitoring tool and does not collect or display EC2 instance metrics.
The option that says: **Using detailed monitoring in CloudWatch** is incorrect because this will typically send metric data of the EC2 instance to CloudWatch in 1-minute periods instead of 5-minute intervals.
The option that says: **Install the AWS X-Ray daemon on the EC2 instance** is incorrect. Although this is helpful for troubleshooting your application, it does not have the capability to track the memory and swap usage of the instance.

References:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/mon-scripts.html
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/monitoring_ec2.html

Check out these Amazon EC2 and CloudWatch Cheat Sheets:
https://tutorialsdojo.com/amazon-elastic-compute-cloud-amazon-ec2/
https://tutorialsdojo.com/amazon-cloudwatch/

27. **27. Question**

Category: CDA – Troubleshooting and Optimization

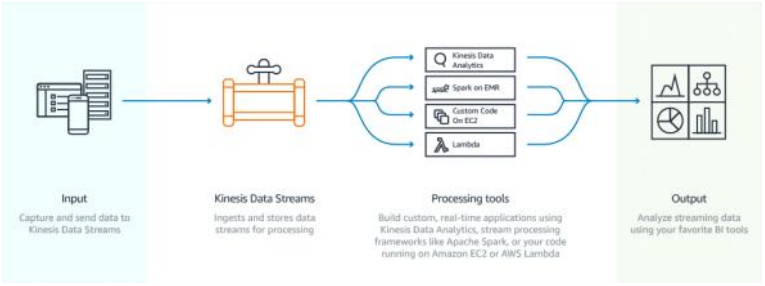
A clickstream application uses Amazon Kinesis Data Stream for real-time processing. `PutRecord` API calls are being used by the producer to send data to the stream. However, there are cases where the producer intermittently restarted while doing the processing, which resulted in sending the same data twice to the stream. This inadvertently causes duplication of entries in the data stream, which affects the processing of the consumers.

Which of the following should you implement to resolve this issue?

- Add more shards.
- Merge shards of the data stream.
- Embed a primary key within the record.
- Split shards of the data stream.

Incorrect

There are two primary reasons why records may be delivered more than one time to your Amazon Kinesis Data Streams application: producer retries and consumer retries. Your application must anticipate and appropriately handle processing individual records multiple times.



Consider a producer that experiences a network-related timeout after it makes a call to `PutRecord`, but before it can receive an acknowledgment from Amazon Kinesis Data Streams. The producer cannot be sure if the record was delivered to Kinesis Data Streams. Assuming that every record is important to the application, the producer would have written to retry the call with the same data. If both `PutRecord` calls on that same data were successfully committed to Kinesis Data Streams, then there will be two Kinesis Data Streams records. Although the two records have identical data, they also have unique sequence numbers. Applications that need strict guarantees should embed a primary key within the record to remove duplicates later when processing. Note that the number of duplicates due to producer retries is usually low compared to the number of duplicates due to consumer retries.

Hence, the correct answer in this scenario is to **embed a primary key within the record** to remove duplicates later when processing.

Adding more shards is incorrect because this is not a suitable solution for handling duplicate records in the Kinesis data stream. This is primarily used to increase the rate of data flowing through the stream.

Splitting shards of the data stream is incorrect because this is used to increase the capacity of the stream and not to avoid any duplicate data.

Merging shards of the data stream is incorrect because this is primarily used to make better use of the unused capacity in the stream and to save on costs.

References:
https://docs.aws.amazon.com/streams/latest/dev/kinesis-record-processor-duplicates.html
https://docs.aws.amazon.com/streams/latest/dev/kinesis-record-processor-scaling.html

Check out this Amazon Kinesis Cheat Sheet:
https://tutorialsdojo.com/amazon-kinesis/

28. **28. Question**

Category: CDA – Troubleshooting and Optimization

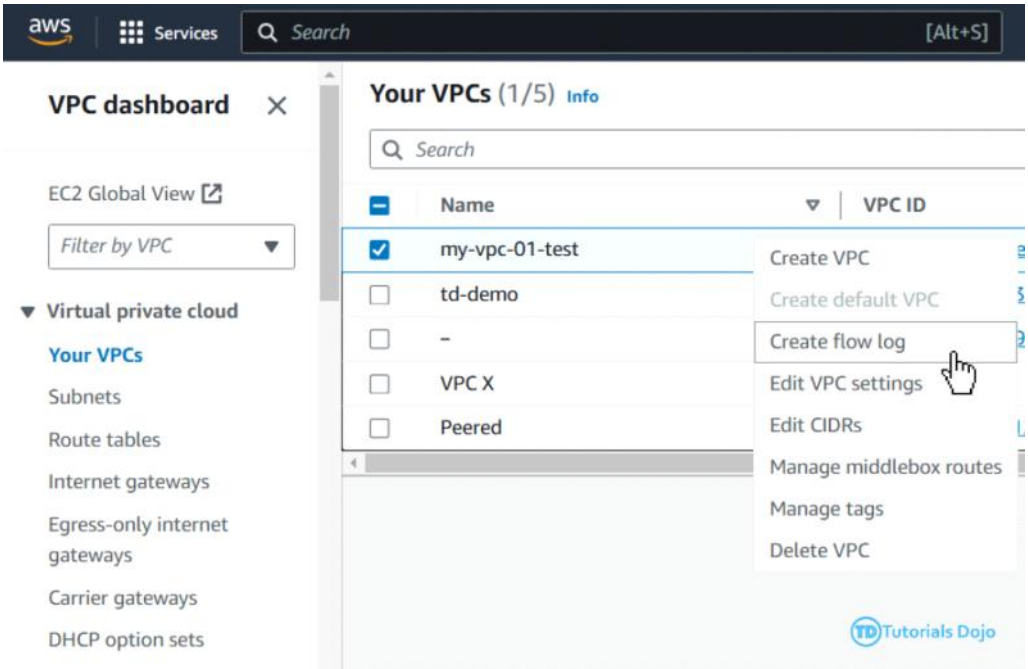
An online stock trading platform is hosted in an Auto Scaling group of EC2 instances with an Application Load Balancer in front to distribute the incoming traffic evenly. The developer must capture information about the IP traffic going to and from network interfaces in your VPC to comply with financial regulatory requirements.

Which of the following options should the developer do to meet the requirement?

- Use CloudTrail logs to track all API calls and capture information about the IP traffic going to and from your VPC.
- Create a flow log in your VPC.
- Install and run the AWS X-Ray daemon to your EC2 instances using an instance metadata script.
- Use AWS Inspector to capture information about the IP traffic going to and from the network interfaces of your EC2 instances.

Incorrect

VPC Flow Logs is a feature that enables you to capture information about the IP traffic going to and from network interfaces in your VPC. Flow log data can be published to Amazon CloudWatch Logs and Amazon S3. After you've created a flow log, you can retrieve and view its data in the chosen destination.



Flow logs can help you with a number of tasks; for example, to troubleshoot why specific traffic is not reaching an instance, which in turn helps you diagnose overly restrictive security group rules. You can also use flow logs as a security tool to monitor the traffic that is reaching your instance. CloudWatch Logs charges apply when using flow logs, whether you send them to CloudWatch Logs or to Amazon S3.

Hence, you should **create a flow log in your VPC** to capture information about the IP traffic going to and from network interfaces in your VPC.

Using CloudTrail logs to track all API calls and capture information about the IP traffic going to and from your VPC is incorrect. Although you can indeed use CloudTrail to track the API call, it can't capture information about the IP traffic of your VPC.

Installing and running the AWS X-Ray daemon to your EC2 instances using an instance metadata script is incorrect because you have to use a user data script and not a metadata. Alternatively, you can instrument your application which is running in an EC2 instance to capture the client's IP address. However, it is much easier to just enable VPC Flow Logs to meet the requirement.

Using AWS Inspector to capture information about the IP traffic going to and from the network interfaces of your EC2 instances is incorrect because this service is just an automated security assessment service that helps

improve the security and compliance of applications deployed on AWS. It doesn't have the ability to capture IP traffic of your VPC.

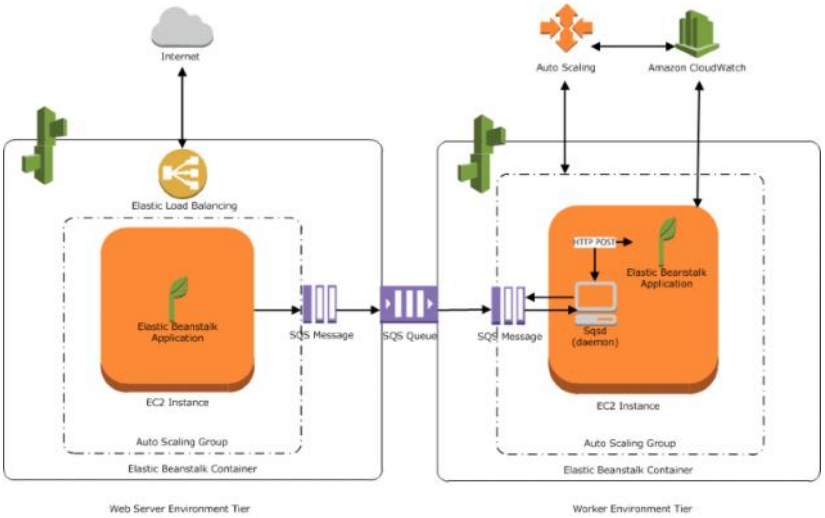
References:
https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-daemon-ec2.html
https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html

Check out these Amazon VPC and AWS X-Ray Cheat Sheets:
https://tutorialsdojo.com/amazon-vpc/
https://tutorialsdojo.com/aws-x-ray/

29. **29. Question**
Category: CDA – Troubleshooting and Optimization
A developer is instructed to configure a worker daemon to queue messages based on a specific schedule using a worker environm ent hosted in Elastic Beanstalk. Periodic tasks should be defined to automatically add jobs to your worker environment's queue at regular intervals.
Which configuration file should the developer add to the source bundle to meet the above requirement?

- `cron.yaml`
- `appspec.yml`
- `Dockerrun.aws.json`
- `env.yaml`

Incorrect
AWS resources created for a worker environment tier include an Auto Scaling group, one or more Amazon EC2 instances, and an IAM role. For the worker environment tier, Elastic Beanstalk also creates and provisions an Amazon SQS queue if you don't already have one. When you launch a worker environment tier, Elastic Beanstalk installs the necessary support files for your programming language of choice and a daemon on each EC2 instance in the Auto Scaling group.
The daemon is responsible for pulling requests from an Amazon SQS queue and then sending the data to the web application running in the worker environment tier that will process those messages. If you have multiple instances in your worker environment tier, each instance has its own daemon, but they all read from the same Amazon SQS queue.



You can define periodic tasks in a file named **cron.yaml** in your source bundle to add jobs to your worker environment's queue automatically at a regular intervals. For example, you can configure and upload a cron.yaml file, which creates two periodic tasks: one that runs every 12 hours and a second that runs at 11 pm UTC every day.
Hence, using the **cron.yaml** is the correct configuration file to be used in this scenario.
`Dockerrun.aws.json` is incorrect because this configuration file is primarily used in multi-container Docker environments that are hosted in Elastic Beanstalk. This can be used alone or combined with source code and content in a source bundle to create an environment on a Docker platform.
`env.yaml` is incorrect because this is primarily used to configure the environment name, solution stack, and environment links to use when creating your environment in Elastic Beanstalk.
`appspec.yml` is incorrect because this is used to manage each application deployment as a series of lifecycle event hooks in CodeDeploy and not in Elastic Beanstalk.

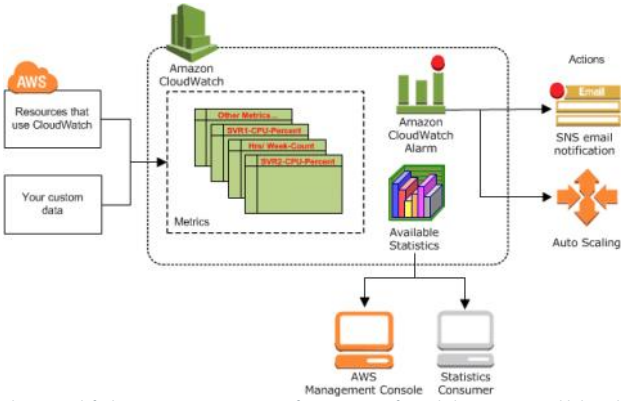
References:
https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features-managing-env-tiers.html#worker-periodictasks
https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/applications-sourcebundle.html

Check out this AWS Elastic Beanstalk Cheat Sheet:
https://tutorialsdojo.com/aws-elastic-beanstalk/

30. **30. Question**
Category: CDA – Troubleshooting and Optimization
A company has 5 different applications running on several On-Demand EC2 instances. The DevOps team is required to set up a graphical representation of the key performance metrics for each application. These system metrics must be available on a single shared screen for more effective and visible monitoring.
Which of the following should the DevOps team do to satisfy this requirement using Amazon CloudWatch?

- Set up a custom CloudWatch namespace with a unique metric name for each application.
- Set up a custom CloudWatch Alarm with a unique metric name for each application.
- Set up a custom CloudWatch Event with a unique metric name for each application.
- Set up a custom CloudWatch dimension with a unique metric name for each application.

Incorrect
Amazon CloudWatch is basically a metrics repository. An AWS service—such as Amazon EC2—puts metrics into the repository, and you retrieve statistics based on those metrics. If you put your own custom metrics into the repository, you can retrieve statistics on these metrics as well.
A *namespace* is a container for CloudWatch metrics. Metrics in different namespaces are isolated from each other so that metrics from different applications are not mistakenly aggregated into the same statistics.



There is no default namespace. You must specify a namespace for each data point you publish to CloudWatch. You can specify a namespace name when you create a metric. These names must contain valid XML characters and be fewer than 256 characters in length. Possible characters are: alphanumeric characters (0-9A-Za-z), period (.), hyphen (-), underscore (_), forward slash (/), hash (#), and colon (:).
The AWS namespaces typically use the following naming convention: `AWS/service`. For example, Amazon EC2 uses the `AWS/EC2` namespace.

Hence, the correct answer is: **Set up a custom CloudWatch namespace with a unique metric name for each application.**
The option that says: **Set up a custom CloudWatch Alarm with a unique metric name for each application** is incorrect because a CloudWatch Alarm simply watches a single metric over a specified time period and performs one or more specified actions based on the value of the metric relative to a threshold over time.
The option that says: **Set up a custom CloudWatch Event with a unique metric name for each application** is incorrect because a CloudWatch Event is primarily used to deliver a near real-time stream of system events that describe changes in Amazon Web Services (AWS) resources, and not for segregating metrics of various applications.
The option that says: **Set up a custom CloudWatch dimension with a unique metric name for each application** is incorrect because a CloudWatch dimension is only a name/value pair that is part of the identity of a metric. You have to use a CloudWatch namespace instead.

References:
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch_concepts.html#Namespace
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/viewing_metrics_with_cloudwatch.html
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/aws-services-cloudwatch-metrics.html

Check out this Amazon CloudWatch Cheat Sheet:
https://tutorialsdojo.com/amazon-cloudwatch/

From <https://portal.tutorialsdojo.com/courses/aws-certified-developer-associate-practice-exams/lessons/practice-exams-section-based-3/quizzes/section-based-troubleshooting-and-optimization-cda/>